



TRƯỜNG ĐẠI HỌC QUY NHƠN

BÀI TIỂU LUẬN CUỐI KỲ HỌC PHẦN PHÂN TÍCH THỐNG KÊ VỚI R

BOOTSTRAP VÀ K-FOLD CROSS-VALIDATION TRONG BÀI TOÁN DỰ ĐOÁN BỆNH GAN

Trình độ đào tạo: Thạc sĩ
Học viên thực hiện: Nguyễn Hồ Bảo Thiên
Lớp: Khoa học dữ liệu K28B
Giảng viên hướng dẫn: TS. Nguyễn Đặng Thiên Thu

Gia Lai, 2026

Mục lục

1	Đặt vấn đề	3
2	Cơ sở lý thuyết	3
2.1	Phương pháp Bootstrap	4
2.2	Phương pháp K-fold Cross-Validation	6
3	Dữ liệu và tiền xử lý	8
3.1	Nguồn dữ liệu và cấu trúc	8
3.2	Thống kê mô tả và giá trị khuyết	9
4	Phân tích khám phá dữ liệu	10
4.1	Phân bố biến mục tiêu	10
4.2	Giới tính và tình trạng bệnh	11
4.3	Tương quan giữa các biến	11
4.4	Đa cộng tuyến và hệ số VIF	12
4.5	Phân bố các biến sinh hoá theo nhóm	14
4.6	Biến đổi log cho các biến lệch phải	14
4.7	Hệ số VIF trên thang log	16
4.8	Thống kê mô tả theo nhóm	18
4.9	Lựa chọn biến	19
5	Thiết kế quy trình mô hình hoá	20
5.1	Tập biến đưa vào mô hình	20
5.2	Chia phân tầng và gán fold	20
5.3	Xử lý mất cân bằng lớp bằng SMOTE	21
5.4	Hồi quy logistic có điều chuẩn ridge	21
5.5	Phòng ngừa rò rỉ dữ liệu	22
5.6	Bộ chỉ số đánh giá	23
6	Ước lượng khoảng tin cậy bằng Bootstrap	24
7	Kiểm định chéo phân tầng	25
8	Tinh chỉnh siêu tham số	27
9	Lựa chọn ngưỡng phân loại	29
10	Đánh giá trên tập kiểm tra	31
10.1	Khoảng tin cậy bootstrap cho toàn bộ chỉ số	32

11 Kết luận	34
11.1 Hạn chế	34
11.2 Hướng mở rộng	35
12 Tài liệu tham khảo	35

1 Đặt vấn đề

Bệnh gan tiến triển âm thầm trong nhiều năm trước khi xuất hiện triệu chứng rõ rệt, và đến khi đó khả năng can thiệp đã thu hẹp đáng kể. Việc sàng lọc sớm dựa trên các chỉ số sinh hoá thường quy như bilirubin, men gan và albumin vì vậy mang ý nghĩa thực tiễn lớn. Bộ dữ liệu *Indian Liver Patient Dataset* (ILPD) ghi lại những chỉ số này cùng chẩn đoán xác định của 583 bệnh nhân, đặt ra một bài toán phân loại nhị phân: dự đoán tình trạng mắc bệnh gan từ kết quả xét nghiệm.

Nếu chỉ dừng ở việc khớp một mô hình rồi công bố một giá trị độ chính xác, bài toán chưa được giải quyết trọn vẹn. Giá trị ấy được tính trên một mẫu hữu hạn và sẽ thay đổi nếu quan sát một nhóm bệnh nhân khác. Hai câu hỏi cần được trả lời là:

1. Trong các cấu hình mô hình có thể xây dựng, cấu hình nào cho khả năng dự đoán tốt nhất trên dữ liệu chưa từng quan sát? Câu hỏi này đòi hỏi một cơ chế đánh giá khách quan, bảo đảm dữ liệu kiểm tra không tham gia vào quá trình lựa chọn. Đó là vai trò của K-fold Cross-Validation.
2. Mô hình được chọn có độ tin cậy đến đâu? Hiệu năng công bố là một ước lượng điểm, cần được bổ sung khoảng tin cậy để phản ánh mức dao động của nó. Đó là vai trò của Bootstrap.

Nói ngắn gọn, K-fold Cross-Validation phục vụ việc lựa chọn mô hình, còn Bootstrap phục vụ việc định lượng độ tin cậy của mô hình đã chọn. Báo cáo trình bày cơ sở lý thuyết của hai phương pháp, sau đó áp dụng chúng vào một quy trình hoàn chỉnh trên bộ dữ liệu ILPD, từ tiền xử lý và lựa chọn biến cho tới tinh chỉnh siêu tham số, lựa chọn ngưỡng phân loại và ước lượng khoảng tin cậy cho hiệu năng cuối cùng.

2 Cơ sở lý thuyết

Bootstrap và K-fold Cross-Validation đều thuộc nhóm kỹ thuật *resampling* (lấy mẫu lại), cùng xuất phát từ một khó khăn nền tảng của thống kê ứng dụng. Nhà nghiên cứu chỉ quan sát được một mẫu hữu hạn rút ra từ tổng thể, trong khi mọi đại lượng tính từ mẫu đó, dù là giá trị trung bình, hệ số hồi quy hay độ chính xác của một mô hình, đều là biến ngẫu nhiên và sẽ nhận giá trị khác nếu rút được một mẫu khác. Câu hỏi cốt lõi vì thế không dừng ở giá trị ước lượng bằng bao nhiêu, mà là ước lượng đó dao động đến mức nào. Khi công thức lý thuyết cho phân phối lấy mẫu không tồn tại hoặc quá phức tạp, các phương pháp *resampling* cho phép mô phỏng chính sự biến thiên ấy từ dữ liệu sẵn có, bằng cách sinh ra nhiều mẫu con và quan sát đại lượng quan tâm thay đổi ra sao qua các mẫu con đó.

2.1 Phương pháp Bootstrap

Bootstrap là phương pháp resampling do Bradley Efron giới thiệu năm 1979, dùng để ước lượng độ chính xác của một đại lượng thống kê, bao gồm sai số chuẩn và khoảng tin cậy, mà không cần giả định về dạng phân phối của tổng thể và không cần thu thập thêm dữ liệu.

Nền tảng lý thuyết. Xét tham số tổng thể $\theta = \theta(F)$, trong đó F là hàm phân phối chưa biết. Từ mẫu quan sát $x_1, \dots, x_n$ ta thu được ước lượng $\hat{\theta} = \theta(\hat{F})$. Muốn đánh giá độ chính xác của $\hat{\theta}$ thì cần biết *phân phối lấy mẫu* của nó, tức tập hợp các giá trị mà $\hat{\theta}$ có thể nhận nếu quá trình rút mẫu kích thước n từ F được lặp lại vô hạn lần. Điều này bất khả thi trên thực tế, bởi F không biết và ta chỉ có duy nhất một mẫu. Ý tưởng của Efron là thay tổng thể chưa biết F bằng *phân phối thực nghiệm* $\hat{F}$, tức phân phối rời rạc gán xác suất $1/n$ cho mỗi quan sát đã có. Theo định lý Glivenko–Cantelli, $\hat{F}$ hội tụ đều về F khi cỡ mẫu tăng, nên với n đủ lớn thì $\hat{F}$ là một thay thế hợp lý cho F . Rút một mẫu kích thước n từ $\hat{F}$ chính là lấy mẫu ngẫu nhiên **có hoàn lại** từ dữ liệu gốc. Đây là nội dung của *nguyên lý plug-in*: phân phối lấy mẫu của $\hat{\theta}$ dưới F được xấp xỉ bằng phân phối của các ước lượng bootstrap $\hat{\theta}^*$ tính dưới $\hat{F}$.

Thuật toán.

1. Từ mẫu gốc kích thước n , lấy mẫu ngẫu nhiên có hoàn lại kích thước n , thu được mẫu bootstrap thứ b .
2. Tính thống kê quan tâm $\hat{\theta}_b^*$ trên mẫu bootstrap đó.
3. Lặp lại hai bước trên B lần, với B thường từ 500 đến vài nghìn, thu được $\hat{\theta}_1^*, \dots, \hat{\theta}_B^*$.
4. Dùng phân phối thực nghiệm của B giá trị này để ước lượng sai số chuẩn và khoảng tin cậy của θ .

Sai số chuẩn và khoảng tin cậy. Sai số chuẩn bootstrap là độ lệch chuẩn của B giá trị thống kê:

$$\widehat{SE}_{boot} = \sqrt{\frac{1}{B-1} \sum_{b=1}^B (\hat{\theta}_b^* - \bar{\theta}^*)^2}, \quad \bar{\theta}^* = \frac{1}{B} \sum_{b=1}^B \hat{\theta}_b^*.$$

Khoảng tin cậy được dựng theo *phương pháp phân vị*. Ở mức tin cậy $1 - \alpha$, hai đầu mút là phân vị mức $\alpha/2$ và $1 - \alpha/2$ của tập B giá trị bootstrap. Chẳng hạn ở mức 95%, khoảng tin cậy trải từ phân vị 2,5% đến phân vị 97,5%.

Ưu điểm.

1. *Không đòi hỏi giả định về phân phối tổng thể.* Các công thức khoảng tin cậy cổ điển thường dựa trên giả định phân phối chuẩn hoặc trên xấp xỉ tiệm cận khi cỡ mẫu lớn. Bootstrap không cần điều kiện nào trong số đó, nên đặc biệt phù hợp với dữ liệu lệch nặng hoặc cỡ mẫu vừa phải, đúng như tình huống của bộ dữ liệu ILPD.
2. *Áp dụng được cho hầu như mọi thống kê.* Nhiều đại lượng quan trọng trong học máy, chẳng hạn recall, precision hay macro F1, không có công thức sai số chuẩn lý thuyết vì chúng là hàm phi tuyến của các tần số đếm được. Bootstrap chỉ cần tính lại đại lượng đó trên mỗi mẫu con, nên xử lý được tất cả các trường hợp này bằng cùng một quy trình.
3. *Không cần thu thập thêm dữ liệu.* Toàn bộ thông tin về mức dao động được khai thác từ chính mẫu đã có. Đây là ưu thế quyết định trong các nghiên cứu y sinh, nơi việc mở rộng cỡ mẫu tốn kém cả về chi phí lẫn thời gian.
4. *Cho biết cả hình dạng phân phối.* Kết quả của bootstrap không chỉ là một cặp đầu mút, mà là toàn bộ tập B giá trị. Nhờ vậy có thể quan sát trực tiếp phân phối lấy mẫu, kiểm tra tính đối xứng cũng như mức độ tập trung của ước lượng.

Hạn chế.

1. *Chi phí tính toán cao.* Toàn bộ quá trình tính thống kê phải lặp lại hàng nghìn lần. Chi phí này còn tăng bội nếu mỗi lần lặp đòi hỏi huấn luyện lại mô hình thay vì chỉ tính lại chỉ số trên các dự đoán có sẵn.
2. *Có thể bị chệch với cỡ mẫu nhỏ.* Phân phối thực nghiệm $\hat{F}$ chỉ xấp xỉ tốt F khi n đủ lớn. Với mẫu nhỏ, mỗi mẫu bootstrap chỉ chứa lại đúng những giá trị đã có, nên khoảng tin cậy thu được có xu hướng hẹp hơn thực tế.
3. *Giả định dữ liệu độc lập và cùng phân phối.* Việc rút mẫu có hoàn lại phá vỡ mọi cấu trúc phụ thuộc giữa các quan sát. Vì vậy phương pháp không áp dụng trực tiếp được cho dữ liệu chuỗi thời gian hay dữ liệu phân cụm theo bệnh viện, theo vùng.
4. *Không tạo ra thông tin mới.* Bootstrap chỉ khai thác lại thông tin có sẵn trong mẫu gốc. Nếu bản thân mẫu gốc không đại diện cho tổng thể, chẳng hạn do sai lệch chọn mẫu, thì khoảng tin cậy thu được vẫn có thể hẹp mà vẫn lệch khỏi giá trị thật.

2.2 Phương pháp K-fold Cross-Validation

K-fold Cross-Validation, hay kiểm định chéo k phần, là kỹ thuật resampling **không hoàn lại**, dùng để ước lượng sai số dự đoán của một mô hình trên dữ liệu chưa từng quan sát, qua đó tránh việc đánh giá mô hình ngay trên chính dữ liệu đã dùng để huấn luyện nó.

Nền tảng lý thuyết. Mục tiêu của một mô hình dự đoán là dự đoán chính xác trên dữ liệu mới, chứ không phải mô tả thật khớp dữ liệu đã có. Nếu đánh giá mô hình ngay trên dữ liệu huấn luyện, ta sẽ thu được ước lượng quá lạc quan, bởi mô hình đã học cả những dao động ngẫu nhiên riêng của mẫu. Đây là hiện tượng *overfitting*. Cách khắc phục đơn giản nhất là tách riêng một phần dữ liệu làm tập kiểm tra, nhưng cách này vừa lãng phí dữ liệu, vừa cho kết quả phụ thuộc mạnh vào một lần chia ngẫu nhiên duy nhất, khiến ước lượng có phương sai lớn khi mẫu nhỏ. K-fold Cross-Validation khắc phục đồng thời cả hai hạn chế bằng cách luân phiên vai trò huấn luyện và kiểm tra qua k lần chia, sao cho mọi quan sát đều được dùng để kiểm tra đúng một lần và dùng để huấn luyện $k - 1$ lần.

Thuật toán.

1. Chia ngẫu nhiên dữ liệu thành k phần rời nhau, kích thước xấp xỉ bằng nhau, gọi là các fold.
2. Với mỗi $i = 1, \dots, k$: huấn luyện mô hình trên $k - 1$ phần còn lại và kiểm tra trên phần thứ i .
3. Tính sai số dự đoán trên từng fold rồi lấy trung bình:

$$CV_{(k)} = \frac{1}{k} \sum_{i=1}^k L\left(y_i, \hat{f}^{(-i)}(x_i)\right),$$

với L là hàm mất mát và $\hat{f}^{(-i)}$ là mô hình được huấn luyện mà không dùng fold thứ i .

Ví dụ minh họa. Xét bài toán phân loại bệnh gan trên ILPD với $n = 583$ quan sát và mô hình hồi quy logistic. Với $k = 5$, dữ liệu được chia thành 5 phần, mỗi phần khoảng 117 quan sát. Ở lần lặp thứ nhất, 4 phần tương ứng khoảng 466 quan sát được dùng để ước lượng các hệ số của mô hình, phần còn lại dùng để dự đoán và tính sai số phân loại. Quá trình lặp lại 5 lần, mỗi phần lần lượt đóng vai tập kiểm tra. Trung bình của 5 sai số này là ước lượng sai số dự đoán, còn độ lệch chuẩn giữa chúng phản ánh mức độ ổn định của mô hình qua các cách chia dữ liệu khác nhau.

Biến thể Stratified k-fold. Khi biến mục tiêu mất cân bằng lớp, việc chia fold hoàn toàn ngẫu nhiên có thể tạo ra những fold có tỉ lệ lớp lệch hẳn so với

tổng thể, thậm chí thiếu vắng lớp thiểu số, làm cho ước lượng sai số bị nhiều. **Stratified k-fold**, hay kiểm định chéo phân tầng, khắc phục bằng cách chia sao cho mỗi fold giữ nguyên tỉ lệ các lớp như trong toàn bộ dữ liệu. Với ILPD, nơi tỉ lệ giữa nhóm mắc bệnh và không mắc bệnh xấp xỉ 71% và 29%, mỗi fold được ràng buộc để duy trì đúng tỉ lệ đó. Nhờ vậy mỗi lần kiểm tra đều phản ánh đúng cơ cấu lớp thực tế và cho ước lượng ổn định hơn. Đây là biến thể được áp dụng trong toàn bộ phần thực nghiệm.

Ưu điểm.

1. *Ước lượng ít lạc quan hơn.* Mọi quan sát đều được dự đoán bởi một mô hình chưa từng sử dụng nó trong huấn luyện. Kết quả thu được vì thế phản ánh khả năng tổng quát hoá chứ không phải mức độ khớp với dữ liệu đã có.
2. *Sử dụng dữ liệu hiệu quả.* Khác với cách tách riêng một tập kiểm tra cố định, ở đây toàn bộ quan sát đều lần lượt tham gia cả vai trò huấn luyện lẫn vai trò kiểm tra. Đây là ưu thế quan trọng khi cỡ mẫu hạn chế.
3. *Cung cấp thước đo độ ổn định.* Ngoài giá trị trung bình, kiểm định chéo còn cho k kết quả riêng lẻ. Độ lệch chuẩn giữa chúng cho biết hiệu năng dao động ra sao khi thay đổi cách chia dữ liệu, một thông tin mà một lần chia train/test đơn lẻ không thể cung cấp.
4. *Không phụ thuộc vào thuật toán cụ thể.* Quy trình chỉ yêu cầu mô hình có thể huấn luyện và dự đoán, nên áp dụng được cho mọi họ mô hình và cho phép so sánh chúng trên cùng một cơ sở.

Hạn chế.

1. *Chi phí tính toán tăng theo k .* Mô hình phải được huấn luyện k lần thay vì một lần. Chi phí này nhân lên khi kiểm định chéo được lồng bên trong một vòng dò siêu tham số, như trong phần thực nghiệm của báo cáo.
2. *Đánh đổi giữa bias và variance theo k .* Với k nhỏ, mỗi mô hình chỉ được huấn luyện trên một phần nhỏ dữ liệu nên ước lượng sai số có xu hướng bị quan. Với k lớn, các tập huấn luyện chồng lấp gần như hoàn toàn nên các kết quả tương quan mạnh với nhau và phương sai của ước lượng trung bình tăng lên.
3. *Các fold không độc lập.* Những tập huấn luyện dùng ở các lần lặp khác nhau chia sẻ phần lớn quan sát. Do đó độ lệch chuẩn giữa các fold không phải là sai số chuẩn đúng nghĩa của ước lượng trung bình, mà chỉ nên hiểu là một chỉ báo về mức độ ổn định.

4. Có thể bị *overfit* nếu dùng lặp lại quá nhiều. Khi cùng một cách chia fold được dùng để so sánh hàng chục cấu hình, cấu hình thắng cuộc có thể chỉ đơn thuần phù hợp với cách chia đó. Kết quả kiểm định chéo khi ấy trở nên lạc quan so với hiệu năng thật trên dữ liệu mới.

3 Dữ liệu và tiền xử lý

3.1 Nguồn dữ liệu và cấu trúc

Bộ dữ liệu ILPD được thu thập tại vùng đông bắc bang Andhra Pradesh, Ấn Độ, gồm 583 hồ sơ bệnh nhân với 10 biến giải thích là tuổi, giới tính và tám chỉ số sinh hoá. Nhãn gốc mã hoá giá trị 1 cho trường hợp có bệnh và giá trị 2 cho trường hợp không bệnh; nhãn này được đổi về quy ước 1 và 0 để thuận tiện cho các công thức đếm ở phần sau.

```
col_names <- c(
  "Age", "Gender", "Total_Bilirubin", "Direct_Bilirubin",
  "Alkaline_Phosphotase", "Alamine_Aminotransferase",
  "Aspartate_Aminotransferase", "Total_Protiens", "Albumin",
  "Albumin_and_Globulin_Ratio", "target"
)

df <- read.csv("Indian Liver Patient Dataset (ILPD).csv",
  header = FALSE, col.names = col_names,
  stringsAsFactors = FALSE)

# Nhãn gốc: 1 = có bệnh, 2 = không bệnh. Đổi về 1 / 0.
df$target <- ifelse(df$target == 2, 0, 1)

cat("Kích thước dữ liệu:", nrow(df), "dòng x", ncol(df), "cột\n")

## Kích thước dữ liệu: 583 dòng x 11 cột

cat("Phân bố target:", sum(df$target == 1), "có bệnh /",
  sum(df$target == 0), "không bệnh\n")

## Phân bố target: 416 có bệnh / 167 không bệnh
```

```
str(df)

## 'data.frame': 583 obs. of 11 variables:
## $ Age : int 65 62 62 58 72 46 26 29 17 55 ...
## $ Gender : chr "Female" "Male" "Male" "Male" ...
## $ Total_Bilirubin : num 0.7 10.9 7.3 1 3.9 1.8 0.9 0.9 0.9 0.7 ...
## $ Direct_Bilirubin : num 0.1 5.5 4.1 0.4 2 0.7 0.2 0.3 0.3 0.2 ...
## $ Alkaline_Phosphotase : int 187 699 490 182 195 208 154 202 202 290 ...
## $ Alamine_Aminotransferase : int 16 64 60 14 27 19 16 14 22 53 ...
## $ Aspartate_Aminotransferase: int 18 100 68 20 59 14 12 11 19 58 ...
## $ Total_Protiens : num 6.8 7.5 7 6.8 7.3 7.6 7 6.7 7.4 6.8 ...
## $ Albumin : num 3.3 3.2 3.3 3.4 2.4 4.4 3.5 3.6 4.1 3.4 ...
## $ Albumin_and_Globulin_Ratio: num 0.9 0.74 0.89 1 0.4 1.3 1 1.1 1.2 1 ...
## $ target : num 1 1 1 1 1 1 1 1 0 1 ...
```

3.2 Thống kê mô tả và giá trị khuyết

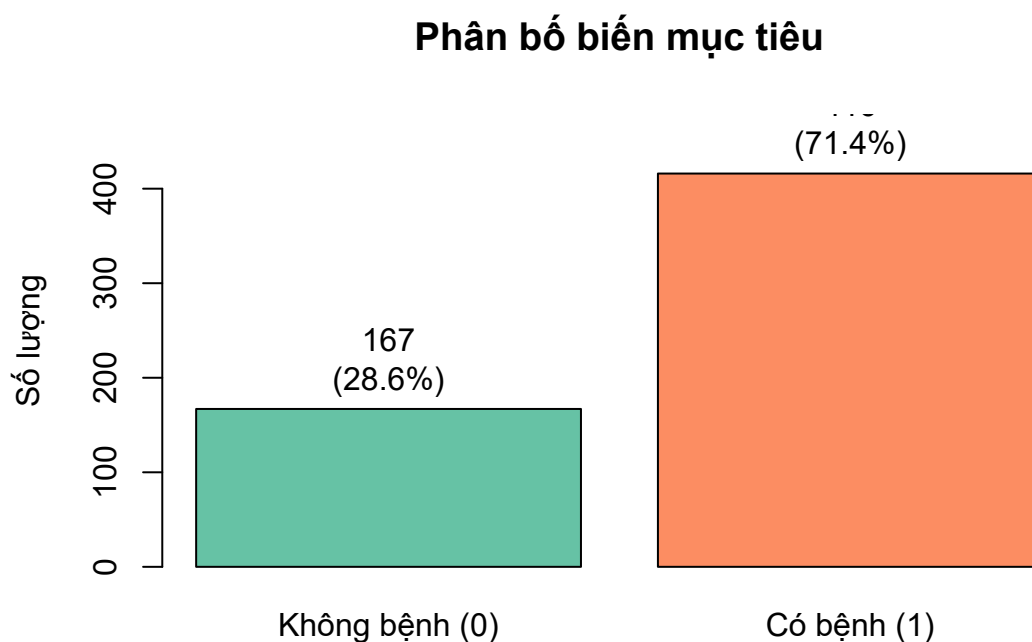
```
summary(df)

##      Age      Gender  Total_Bilirubin  Direct_Bilirubin
## Min.   : 4.00   Length :583   Min.    : 0.400   Min.    : 0.100
## 1st Qu.:33.00   N.unique : 2   1st Qu.: 0.800   1st Qu.: 0.200
## Median :45.00   N.blank  : 0   Median : 1.000   Median : 0.300
## Mean   :44.75   Min.nchar: 4   Mean    : 3.299   Mean    : 1.486
## 3rd Qu.:58.00   Max.nchar: 6   3rd Qu.: 2.600   3rd Qu.: 1.300
## Max.   :90.00           Max.    :75.000   Max.    :19.700
##
## Alkaline_Phosphotase Alamine_Aminotransferase
## Min.    : 63.0      Min.    : 10.00
## 1st Qu.: 175.5      1st Qu.: 23.00
## Median : 208.0      Median : 35.00
## Mean    : 290.6      Mean    : 80.71
## 3rd Qu.: 298.0      3rd Qu.: 60.50
## Max.    :2110.0      Max.    :2000.00
##
## Aspartate_Aminotransferase Total_Protiens      Albumin
## Min.    : 10.0      Min.    :2.700   Min.    :0.900
## 1st Qu.: 25.0      1st Qu.:5.800   1st Qu.:2.600
## Median : 42.0      Median :6.600   Median :3.100
## Mean    :109.9      Mean    :6.483   Mean    :3.142
## 3rd Qu.: 87.0      3rd Qu.:7.200   3rd Qu.:3.800
## Max.    :4929.0      Max.    :9.600   Max.    :5.500
##
## Albumin_and_Globulin_Ratio      target
## Min.    :0.3000      Min.    :0.0000
## 1st Qu.:0.7000      1st Qu.:0.0000
## Median :0.9300      Median :1.0000
## Mean    :0.9471      Mean    :0.7136
## 3rd Qu.:1.1000      3rd Qu.:1.0000
## Max.    :2.8000      Max.    :1.0000
## NAs      :4

colSums(is.na(df))

##      Age      Gender
##      0      0
## Total_Bilirubin  Direct_Bilirubin
##      0      0
## Alkaline_Phosphotase Alamine_Aminotransferase
##      0      0
## Aspartate_Aminotransferase      Total_Protiens
##      0      0
## Albumin Albumin_and_Globulin_Ratio
##      0      4
## target
##      0
```

Toàn bộ dữ liệu chỉ có 4 giá trị khuyết, đều nằm ở biến `Albumin_and_Globulin_Ratio`. Con số này rất nhỏ so với 583 quan sát nên cách xử lý gần như không ảnh hưởng tới kết quả. Dù vậy, các giá trị khuyết vẫn được điền bằng trung vị thay vì loại bỏ quan sát, và điều quan trọng hơn là việc điền khuyết được đặt bên trong quy trình huấn luyện để trung vị chỉ được tính từ tập huấn luyện. Lý do của yêu cầu này được trình bày ở Mục 5.5.



Hình 1: Phân bố biến mục tiêu

4 Phân tích khám phá dữ liệu

4.1 Phân bố biến mục tiêu

```
counts <- table(df$target)
pct <- round(100 * counts / sum(counts), 1)

bp <- barplot(counts,
  names.arg = c("Không bệnh (0)", "Có bệnh (1)"),
  col = c("#66c2a5", "#fc8d62"),
  main = "Phân bố biến mục tiêu", ylab = "Số lượng",
  ylim = c(0, max(counts) * 1.15))
text(bp, counts, labels = paste0(counts, "\n(", pct, "%)"), pos = 3)
```

Tỉ lệ 71,4% và 28,6% cho thấy dữ liệu mất cân bằng ở mức trung bình. Hệ quả trực tiếp là độ chính xác tổng thể trở thành một chỉ số gây hiểu nhầm, bởi một mô hình dự đoán tất cả bệnh nhân đều mắc bệnh đã đạt độ chính xác 0,714 mà không mang lại thông tin nào. Vì vậy toàn bộ phần đánh giá về sau sử dụng recall và precision của từng lớp cùng với macro F1, thay vì dựa vào độ chính xác tổng thể.

4.2 Giới tính và tình trạng bệnh

Trước khi đưa biến Gender vào mô hình, cần kiểm tra xem biến này có thực sự liên hệ với biến mục tiêu hay không. Giả thuyết H_0 được đặt ra là tỉ lệ mắc bệnh không khác nhau giữa hai giới.

```
contingency <- table(df$Gender, df$target)
print(contingency)

##
##           0    1
## Female  50   92
## Male   117  324

chi_res <- chisq.test(contingency)
cat("\nChi-square =", round(chi_res$statistic, 4),
    "| p-value =", round(chi_res$p.value, 4), "\n")

##
## Chi-square = 3.5466 | p-value = 0.0597

cat("Bảng tần số kỳ vọng (cần >= 5 để kiểm định hợp lệ):\n")

## Bảng tần số kỳ vọng (cần >= 5 để kiểm định hợp lệ):

print(round(chi_res$expected, 2))

##
##           0    1
## Female  40.68 101.32
## Male   126.32 314.68
```

Tần số kỳ vọng ở cả bốn ô đều lớn hơn 5 nên kiểm định hợp lệ. Với $p = 0,0597$, lớn hơn mức ý nghĩa 0,05, chưa đủ bằng chứng để bác bỏ H_0 : dữ liệu không cho thấy khác biệt có ý nghĩa thống kê về tỉ lệ mắc bệnh giữa nam và nữ. Cần lưu ý rằng không bác bỏ được H_0 khác với chứng minh H_0 đúng. Giá trị p ở đây nằm sát ngưỡng, nên kết luận hợp lý là bằng chứng còn yếu, chứ không phải mối liên hệ chắc chắn không tồn tại.

4.3 Tương quan giữa các biến

```
numeric_cols <- c(
  "Age", "Total_Bilirubin", "Direct_Bilirubin", "Alkaline_Phosphotase",
  "Alamine_Aminotransferase", "Aspartate_Aminotransferase",
  "Total_Protiens", "Albumin", "Albumin_and_Globulin_Ratio"
)

corr_mat <- cor(df[, c(numeric_cols, "target")], use = "complete.obs")

# Rút gọn tên khi in để bảng vừa khổ giấy
corr_disp <- round(corr_mat, 2)
dimnames(corr_disp) <- list(abbreviate(rownames(corr_mat), 12),
                           abbreviate(colnames(corr_mat), 6))
print(corr_disp)

##
##      Age  Ttl_Bl Drct_B Alkl_P Almn_A Aspr_A Ttl_Pr Albumn
## Age      1.00  0.01  0.01  0.08 -0.09 -0.02 -0.19 -0.26
## Total_Bilrbn 0.01  1.00  0.87  0.21  0.21  0.24 -0.01 -0.22
## Direct_Blrbn 0.01  0.87  1.00  0.23  0.23  0.26  0.00 -0.23
## Alkln_Phspt 0.08  0.21  0.23  1.00  0.12  0.17 -0.03 -0.16
```

```
## Almn_Amntns -0.09 0.21 0.23 0.12 1.00 0.79 -0.04 -0.03
## Asprtt_Amntn -0.02 0.24 0.26 0.17 0.79 1.00 -0.03 -0.08
## Total_Protns -0.19 -0.01 0.00 -0.03 -0.04 -0.03 1.00 0.78
## Albumin -0.26 -0.22 -0.23 -0.16 -0.03 -0.08 0.78 1.00
## Albmnd_G_R -0.22 -0.21 -0.20 -0.23 0.00 -0.07 0.23 0.69
## target 0.13 0.22 0.25 0.18 0.16 0.15 -0.03 -0.16
##
## A_G_R target
## Age -0.22 0.13
## Total_Bilrnb -0.21 0.22
## Direct_Bilrnb -0.20 0.25
## Alkln_Phspht -0.23 0.18
## Almn_Amntns 0.00 0.16
## Asprtt_Amntn -0.07 0.15
## Total_Protns 0.23 -0.03
## Albumin 0.69 -0.16
## Albmnd_G_R 1.00 -0.16
## target -0.16 1.00
```

```
par(mar = c(9, 9, 3, 2))
image(seq_len(ncol(corr_mat)), seq_len(nrow(corr_mat)),
      t(corr_mat[nrow(corr_mat):1, ]),
      col = colorRampPalette(c("#3288bd", "white", "#d53e4f"))(64),
      zlim = c(-1, 1), axes = FALSE, xlab = "", ylab = "",
      main = "Ma trận tương quan")
axis(1, seq_len(ncol(corr_mat)), colnames(corr_mat), las = 2, cex.axis = 0.7)
axis(2, seq_len(nrow(corr_mat)), rev(rownames(corr_mat)), las = 2,
      cex.axis = 0.7)
```

```
par(mar = c(5, 4, 4, 2) + 0.1)
```

Ba cặp biến có tương quan rất cao. Cặp thứ nhất là Total_Bilirubin với Direct_Bilirubin ($r = 0,87$), cặp thứ hai là hai men gan Alamine_Aminotransferase và Aspartate_Aminotransferase ($r = 0,79$), cặp thứ ba là Total_Protiens với Albumin ($r = 0,78$). Các quan hệ này phù hợp với bản chất sinh hoá của những chỉ số đó, khi bilirubin trực tiếp là một thành phần của bilirubin toàn phần và albumin là thành phần chính của protein toàn phần. Tuy nhiên chúng lại gây khó khăn cho hồi quy logistic, bởi các biến gần như trùng thông tin sẽ làm ma trận thiết kế gần suy biến và khiến hệ số ước lượng mất ổn định.

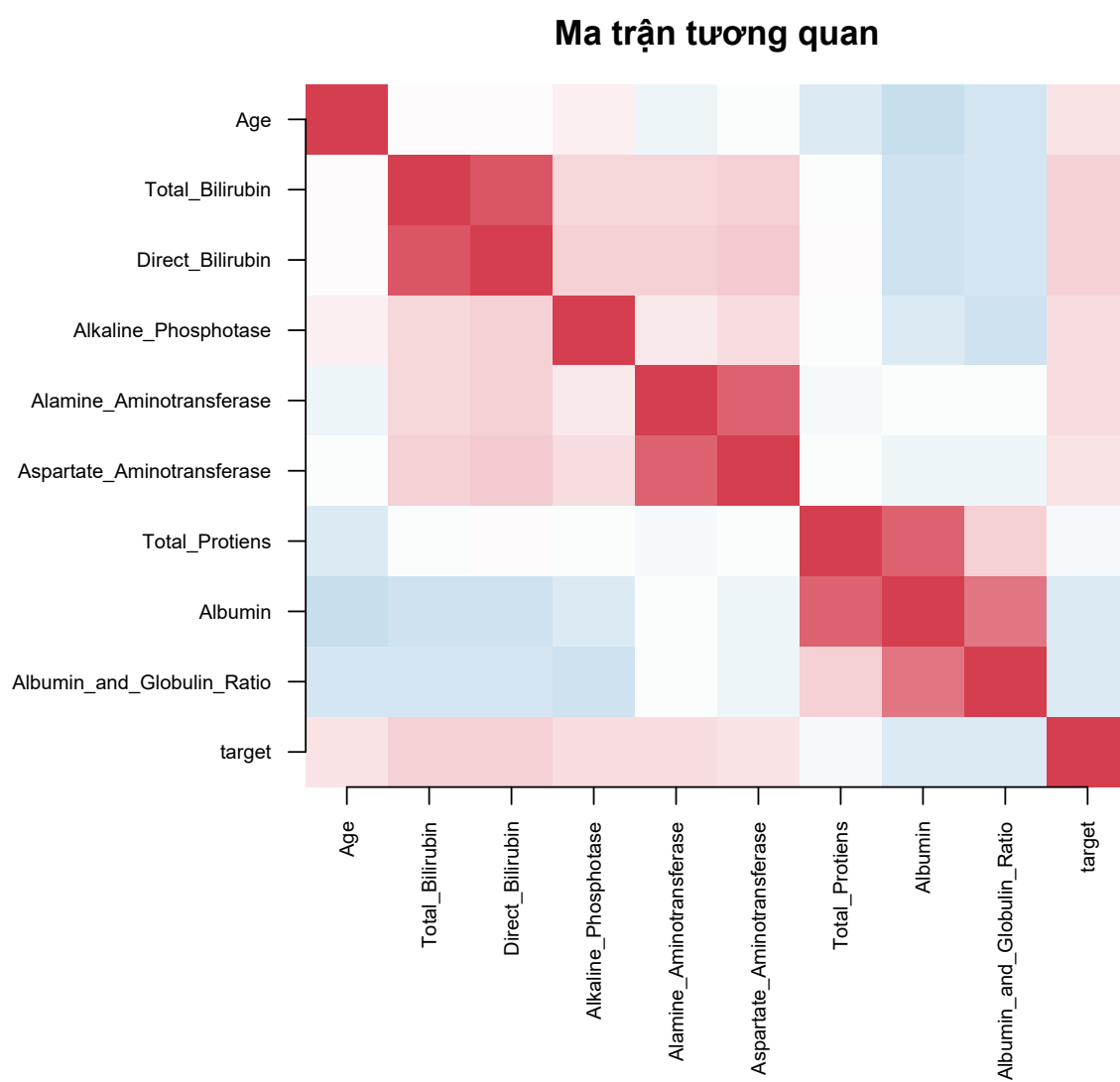
Ở chiều ngược lại, tương quan giữa từng biến với biến mục tiêu đều yếu, giá trị lớn nhất chỉ đạt 0,25. Điều này cho thấy không có biến đơn lẻ nào tách được hai nhóm một cách rõ ràng.

4.4 Đa cộng tuyến và hệ số VIF

Hệ số tương quan chỉ phát hiện được quan hệ giữa từng cặp biến. Hệ số phóng đại phương sai (Variance Inflation Factor) đo mức độ một biến bị giải thích bởi toàn bộ các biến còn lại:

$$\text{VIF}_j = \frac{1}{1 - R_j^2},$$

với R_j^2 là hệ số xác định của hồi quy biến j lên tất cả các biến khác.



Hình 2: Ma trận tương quan giữa các biến sinh hoá và biến mục tiêu

```
compute_vif <- function(data) {
  vars <- colnames(data)
  sapply(vars, function(v) {
    others <- setdiff(vars, v)
    fml <- as.formula(paste(v, "~", paste(others, collapse = " + ")))
    r2 <- summary(lm(fml, data = data))$r.squared
    1 / (1 - r2)
  })
}

X_vif <- df[, c(numeric_cols, "Gender")]
X_vif$Gender <- ifelse(X_vif$Gender == "Male", 1, 0)
X_vif$Albumin_and_Globulin_Ratio[is.na(X_vif$Albumin_and_Globulin_Ratio)] <-
  median(X_vif$Albumin_and_Globulin_Ratio, na.rm = TRUE)

vif_raw <- compute_vif(X_vif)
print(round(sort(vif_raw, decreasing = TRUE), 3))

##           Albumin           Total_Protiens
##           10.124           5.549
##      Direct_Bilirubin      Total_Bilirubin
##           4.525           4.277
## Albumin_and_Globulin_Ratio      Alamine_Aminotransferase
##           3.683           2.820
## Aspartate_Aminotransferase      Alkaline_Phosphotase
##           2.811           1.122
##           Age           Gender
##           1.099           1.034
```

Theo quy ước thường dùng, giá trị VIF lớn hơn 5 là đáng lo ngại và lớn hơn 10 là nghiêm trọng. Biến Albumin vượt ngưỡng 10 còn Total_Protiens vượt ngưỡng 5, đúng như cặp tương quan 0,78 đã gợi ý.

4.5 Phân bố các biến sinh hoá theo nhóm

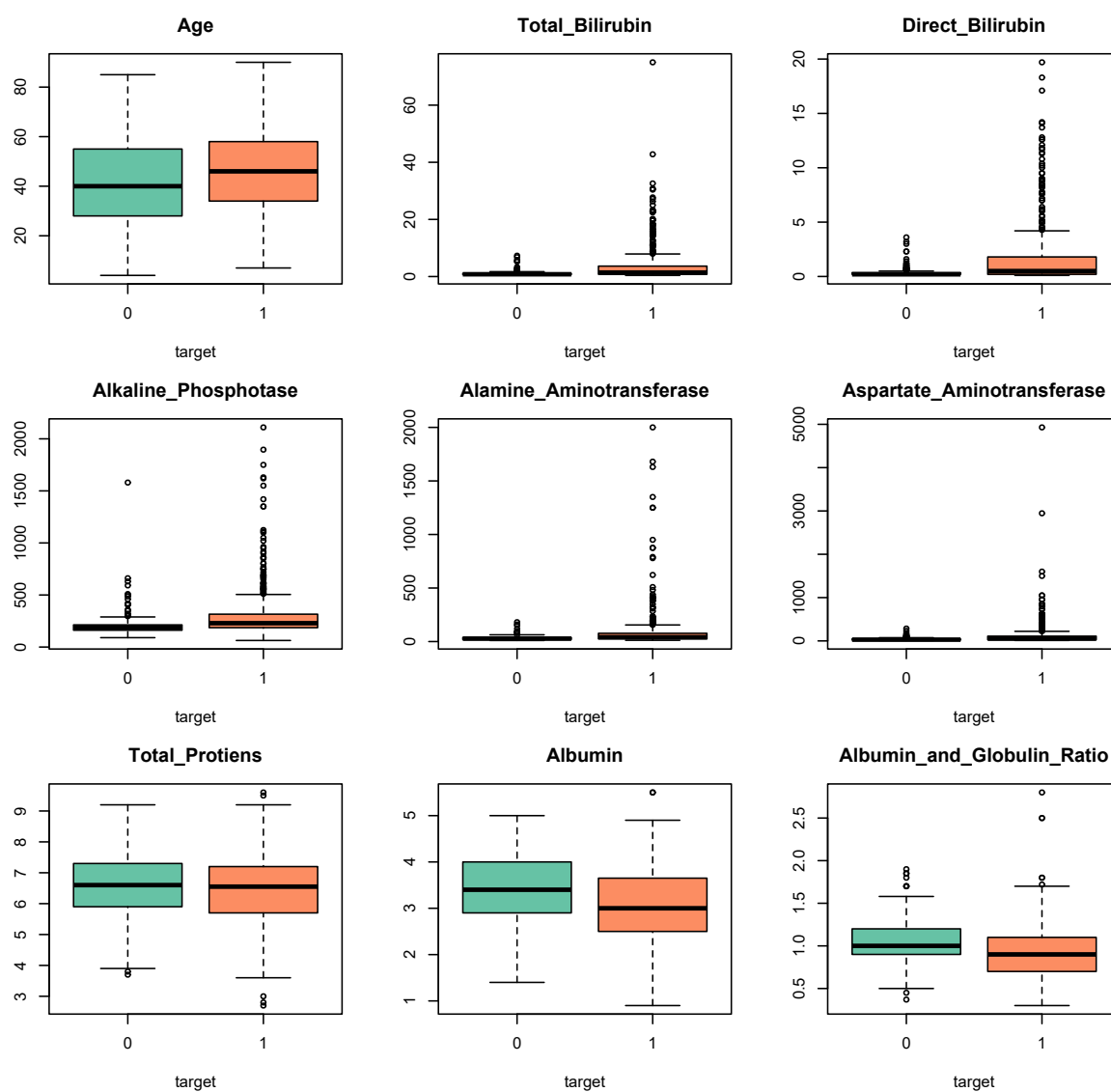
```
par(mfrow = c(3, 3), mar = c(4, 4, 3, 1))
for (col in numeric_cols) {
  boxplot(df[[col]] ~ df$target, col = c("#66c2a5", "#fc8d62"),
    main = col, xlab = "target", ylab = "")
}
```

```
par(mfrow = c(1, 1), mar = c(5, 4, 4, 2) + 0.1)
```

Hình 3 cho thấy năm biến gồm hai chỉ số bilirubin và ba men gan đều lệch phải rất nặng, với phân hộp bị nén sát trục còn các quan sát cực trị kéo dài lên trên. Đây là dạng phân bố điển hình của các chỉ số men gan.

4.6 Biến đổi log cho các biến lệch phải

Phép biến đổi $\log(1 + x)$ được áp dụng cho năm biến lệch phải nặng nói trên. Phép biến đổi này nén đuôi phân phối nhưng vẫn bảo toàn thứ hạng của các quan sát, do là hàm đơn điệu tăng. Việc bảo toàn thứ hạng có hai ý nghĩa. Thứ nhất, các giá trị cực đại vẫn giữ được vai trò tín hiệu lâm sàng của chúng, chẳng hạn nồng độ men gan AST khoảng 5000 U/L tương ứng với suy gan cấp, thay vì bị



Hình 3: Phân bố các biến sinh hoá theo nhóm bệnh trên thang đo gốc

keo lại và mất đi khả năng phân biệt hai nhóm. Thứ hai, kết quả của các kiểm định dựa trên thứ hạng ở phần sau không bị thay đổi bởi phép biến đổi.

```
skewed_cols <- c("Total_Bilirubin", "Direct_Bilirubin",
                "Alkaline_Phosphotase", "Alamine_Aminotransferase",
                "Aspartate_Aminotransferase")

df_raw <- df # giữ bản gốc để tra thang đo lâm sàng
for (col in skewed_cols) df[[col]] <- log1p(df[[col]])

# Hệ số bất đối xứng (skewness)
skewness <- function(x) {
  x <- x[!is.na(x)]
  mean((x - mean(x))^3) / (mean((x - mean(x))^2)^1.5)
}

print(round(data.frame(
  goc      = sapply(df_raw[skewed_cols], skewness),
  sau_log1p = sapply(df[skewed_cols], skewness)
), 2))

##                goc sau_log1p
## Total_Bilirubin      4.89    1.72
## Direct_Bilirubin     3.20    1.68
## Alkaline_Phosphotase  3.76    1.33
## Alamine_Aminotransferase 6.53    1.47
## Aspartate_Aminotransferase 10.52    1.23
```

Hiệu quả của phép biến đổi rất rõ rệt. Biến Aspartate_Aminotransferase giảm hệ số bất đối xứng từ 10,52 xuống 1,23, biến Alamine_Aminotransferase giảm từ 6,53 xuống 1,47. Cả năm biến đều trở về mức bất đối xứng nhẹ, và khác biệt giữa hai nhóm trở nên quan sát được rõ ràng hơn (Hình 4).

```
par(mfrow = c(3, 3), mar = c(4, 4, 3, 1))
for (col in numeric_cols) {
  boxplot(df[[col]] ~ df$target, col = c("#66c2a5", "#fc8d62"),
    main = if (col %in% skewed_cols) paste0("log1p(", col, ")") else col,
    xlab = "target", ylab = "")
}
```

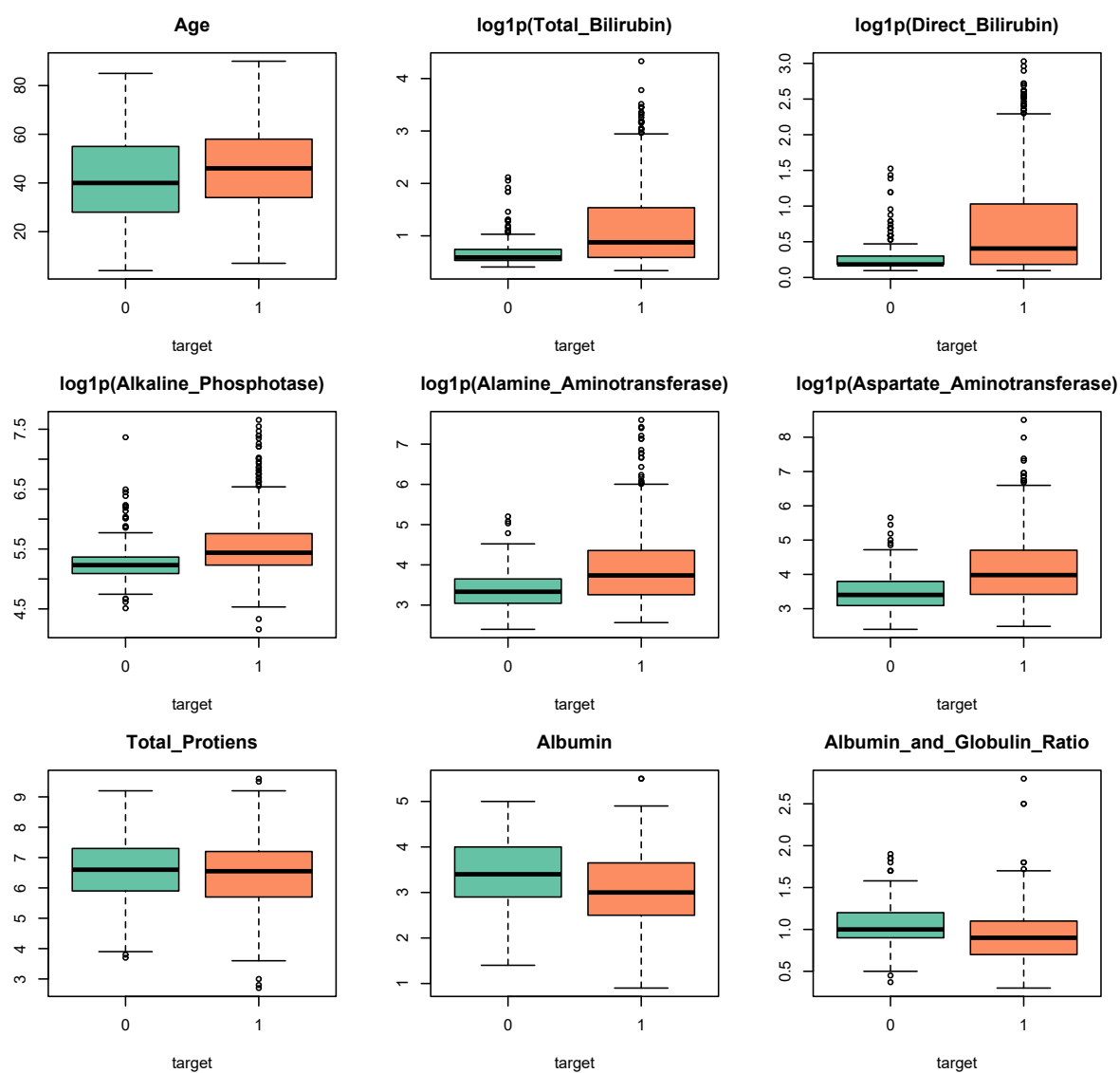
```
par(mfrow = c(1, 1), mar = c(5, 4, 4, 2) + 0.1)
```

4.7 Hệ số VIF trên thang log

Một điểm dễ bị bỏ qua là hệ số tương quan Pearson và hệ số VIF không bất biến với phép biến đổi log, khác với các kiểm định dựa trên thứ hạng. Ma trận thiết kế mà mô hình thực sự sử dụng là ma trận trên thang log, nên VIF cần được tính lại trên đúng thang đo đó.

```
X_vif_log <- X_vif
for (col in skewed_cols) X_vif_log[[col]] <- log1p(df_raw[[col]])
vif_log <- compute_vif(X_vif_log)

print(round(data.frame(
  VIF_goc      = vif_raw,
  VIF_log      = vif_log[names(vif_raw)],
  chênh_lech   = vif_log[names(vif_raw)] - vif_raw
)[order(-vif_log[names(vif_raw)]), ], 2))
```



Hình 4: Phân bố các biến sinh hoá theo nhóm bệnh sau biến đổi log

##	VIF_goc	VIF_log	chenh_lech
## Total_Bilirubin	4.28	20.72	16.44
## Direct_Bilirubin	4.53	20.55	16.03
## Albumin	10.12	10.62	0.49
## Total_Protiens	5.55	5.65	0.10
## Aspartate_Aminotransferase	2.81	4.18	1.37
## Alamine_Aminotransferase	2.82	3.94	1.12
## Albumin_and_Globulin_Ratio	3.68	3.75	0.07
## Alkaline_Phosphotase	1.12	1.28	0.16
## Age	1.10	1.10	0.00
## Gender	1.03	1.06	0.03

Kết quả thay đổi hẳn nhận định ban đầu. Trên thang gốc, hai biến bilirubin có VIF chỉ khoảng 4,3 đến 4,5, tức dưới ngưỡng cảnh báo. Trên thang log, giá trị này tăng lên trên 20, tức mức nghiêm trọng. Nguyên nhân là quan hệ giữa hai biến vốn có dạng nhân tính, và phép lấy log đã chuyển nó thành quan hệ tuyến tính gần như hoàn hảo. Nếu chỉ căn cứ vào VIF trên thang gốc, ta sẽ kết luận sai rằng cặp bilirubin không có vấn đề. Đây là cơ sở trực tiếp để loại biến Direct_Bilirubin khỏi mô hình.

4.8 Thống kê mô tả theo nhóm

Bảng thống kê dưới đây được tính trên thang đo gốc, để các con số giữ nguyên đơn vị lâm sàng và có thể đối chiếu với khoảng tham chiếu xét nghiệm.

```
for (col in numeric_cols) {
  s <- tapply(df_raw[[col]], df_raw$target, function(x) {
    x <- x[!is.na(x)]
    c(min = min(x), max = max(x), mean = mean(x),
      median = median(x), sd = sd(x))
  })
  cat("\n", col, "\n")
  print(round(do.call(rbind, s), 2))
}
```

```
##
## Age
## min max mean median sd
## 0 4 85 41.24 40 17.00
## 1 7 90 46.15 46 15.65
##
## Total_Bilirubin
## min max mean median sd
## 0 0.5 7.3 1.14 0.8 1.00
## 1 0.4 75.0 4.16 1.4 7.14
##
## Direct_Bilirubin
## min max mean median sd
## 0 0.1 3.6 0.40 0.2 0.52
## 1 0.1 19.7 1.92 0.5 3.21
##
## Alkaline_Phosphotase
## min max mean median sd
## 0 90 1580 219.75 186 140.99
## 1 63 2110 319.01 229 268.31
##
## Alamine_Aminotransferase
## min max mean median sd
## 0 10 181 33.65 27 25.06
## 1 12 2000 99.61 41 212.77
##
## Aspartate_Aminotransferase
## min max mean median sd
## 0 10 285 40.69 29.0 36.41
## 1 11 4929 137.70 52.5 337.39
##
## Total_Protiens
## min max mean median sd
## 0 3.7 9.2 6.54 6.60 1.06
## 1 2.7 9.6 6.46 6.55 1.09
##
## Albumin
## min max mean median sd
## 0 1.4 5.0 3.34 3.4 0.78
```

```
## 1 0.9 5.5 3.06 3.0 0.79
##
## Albumin_and_Globulin_Ratio
## min max mean median sd
## 0 0.37 1.9 1.03 1.0 0.29
## 1 0.30 2.8 0.91 0.9 0.33
```

Chênh lệch giữa trung bình và trung vị ở nhóm bệnh rất lớn. Chẳng hạn men gan AST có trung bình 137,70 trong khi trung vị chỉ 52,50. Điều này một lần nữa khẳng định phân phối lệch nặng và củng cố lựa chọn sử dụng kiểm định phi tham số ở bước tiếp theo.

4.9 Lựa chọn biến

Do phần lớn biến lệch phải nặng, việc so sánh hai nhóm được thực hiện bằng kiểm định phi tham số Mann-Whitney U, không đòi hỏi giả định về dạng phân phối. Kích thước ảnh hưởng đi kèm là hệ số tương quan *rank-biserial*, nhận giá trị trong đoạn $[-1, 1]$, với giá trị tuyệt đối càng lớn thì mức tách biệt giữa hai nhóm càng rõ. Riêng biến Gender là biến phân loại nên sử dụng kết quả kiểm định Chi-square ở trên, với kích thước ảnh hưởng là hệ số Cramér's V.

Vì mười kiểm định được thực hiện đồng thời ở mức $\alpha = 0,05$, xác suất có ít nhất một kết quả dương tính giả không còn là 5%. Thủ tục Benjamini–Hochberg được áp dụng để hiệu chỉnh giá trị p , qua đó kiểm soát tỉ lệ phát hiện sai (False Discovery Rate).

```
fs <- data.frame(feature = character(), p_value = numeric(),
                 effect_size = numeric(), stringsAsFactors = FALSE)

for (col in numeric_cols) {
  g0 <- df[[col]][df$target == 0]
  g1 <- df[[col]][df$target == 1]
  w <- wilcox.test(g0, g1, exact = FALSE)
  n0 <- sum(!is.na(g0)); n1 <- sum(!is.na(g1))
  rb <- 1 - 2 * w$statistic / (n0 * n1) # rank-biserial correlation
  fs <- rbind(fs, data.frame(feature = col, p_value = w$p.value,
                             effect_size = as.numeric(rb)))
}

# Gender: dùng Cramer's V làm kích thước ảnh hưởng
n_total <- sum(contingency)
cramers_v <- sqrt(chi_res$statistic /
                  (n_total * (min(dim(contingency)) - 1)))
fs <- rbind(fs, data.frame(feature = "Gender", p_value = chi_res$p.value,
                           effect_size = as.numeric(cramers_v)))

fs$p_fdr <- p.adjust(fs$p_value, method = "BH") # Benjamini-Hochberg
fs$significant <- fs$p_fdr < 0.05
fs <- fs[order(fs$p_value), ]

fs_disp <- fs
fs_disp$p_value <- signif(fs_disp$p_value, 3)
fs_disp$effect_size <- round(fs_disp$effect_size, 3)
fs_disp$p_fdr <- signif(fs_disp$p_fdr, 3)
print(fs_disp, row.names = FALSE)

##
## feature p_value effect_size p_fdr significant
## Aspartate_Aminotransferase 9.21e-14 0.394 9.21e-13 TRUE
## Total_Bilirubin 2.29e-13 0.387 1.14e-12 TRUE
## Direct_Bilirubin 7.43e-13 0.372 2.48e-12 TRUE
```

##	Alamine_Aminotransferase	2.33e-12	0.371	5.83e-12	TRUE
##	Alkaline_Phosphotase	4.35e-11	0.349	8.69e-11	TRUE
##	Albumin_and_Globulin_Ratio	5.81e-06	-0.240	9.69e-06	TRUE
##	Albumin	5.57e-05	-0.213	7.95e-05	TRUE
##	Age	1.77e-03	0.165	2.22e-03	TRUE
##	Gender	5.97e-02	0.078	6.63e-02	FALSE
##	Total_Protiens	4.37e-01	-0.041	4.37e-01	FALSE

Tám trên mười biến đạt ý nghĩa thống kê sau hiệu chỉnh FDR. Hai biến bị loại là Gender với $p_{FDR} = 0,066$ và Total_Protiens với $p_{FDR} = 0,437$. Nhóm men gan và bilirubin có kích thước ảnh hưởng lớn nhất, trong khoảng 0,35 đến 0,39. Biến Age tuy đạt ý nghĩa thống kê nhưng kích thước ảnh hưởng chỉ 0,17, cho thấy ý nghĩa thống kê và mức độ quan trọng thực tế là hai khía cạnh khác nhau.

5 Thiết kế quy trình mô hình hoá

5.1 Tập biến đưa vào mô hình

Từ 10 biến ban đầu, ba biến bị loại với lý do khác nhau. Direct_Bilirubin bị loại do trùng thông tin với Total_Bilirubin, thể hiện qua hệ số tương quan 0,87 và VIF trên thang log khoảng 20. Gender và Total_Protiens bị loại do không đạt ngưỡng ý nghĩa sau hiệu chỉnh FDR.

```
model_cols <- c("Age", "Total_Bilirubin", "Alkaline_Phosphotase",
               "Alamine_Aminotransferase", "Aspartate_Aminotransferase",
               "Albumin", "Albumin_and_Globulin_Ratio")

X_all <- as.matrix(df[, model_cols])
y_all <- df$target
```

Cần thừa nhận rằng đa cộng tuyến chưa được loại bỏ hoàn toàn. Hai biến Albumin và Albumin_and_Globulin_Ratio vẫn liên quan chặt về mặt định nghĩa, còn cặp men gan ALT và AST vẫn giữ mức tương quan 0,79. Đây là một lý do nữa để sử dụng hồi quy có điều chuẩn ridge ở Mục 5.4.

5.2 Chia phân tầng và gán fold

Hai thủ tục dưới đây đều xáo trộn trong nội bộ từng lớp trước khi chia, nhờ vậy tỉ lệ 71 trên 29 được giữ nguyên ở mọi tập con.

```
stratified_split <- function(y, test_frac = 0.2, seed = 42) {
  set.seed(seed)
  test_idx <- integer(0)
  for (cls in sort(unique(y))) {
    idx <- which(y == cls)
    test_idx <- c(test_idx, sample(idx, round(length(idx) * test_frac)))
  }
  list(train = setdiff(seq_along(y), test_idx), test = sort(test_idx))
}
```

```
stratified_folds <- function(y, k = 5, seed = 42) {
  set.seed(seed)
  fold <- integer(length(y))
  for (cls in sort(unique(y))) {
    idx <- sample(which(y == cls))
    fold[idx] <- rep_len(seq_len(k), length(idx))
  }
  fold
}
```

5.3 Xử lý mất cân bằng lớp bằng SMOTE

Với tỉ lệ lớp 71 trên 29, mô hình huấn luyện trực tiếp sẽ thiên vị lớp đa số. Kỹ thuật SMOTE (Synthetic Minority Over-sampling Technique) cân bằng lại bằng cách sinh thêm mẫu tổng hợp cho lớp thiểu số, thay vì nhân bản mẫu sẵn có. Nhân bản chỉ làm tăng trọng số của đúng những điểm đã tồn tại và dễ dẫn tới overfitting. Mỗi mẫu mới được nội suy tuyến tính giữa một quan sát thiểu số x_a và một trong k láng giềng gần nhất cùng lớp của nó:

$$x_{new} = x_a + u \cdot (x_b - x_a), \quad u \sim \mathcal{U}(0, 1),$$

tức là một điểm ngẫu nhiên nằm trên đoạn thẳng nối hai quan sát thật.

```
smote_oversample <- function(X, y, k = 5, seed = 42) {
  set.seed(seed)
  tab <- table(y)
  min_lab <- as.numeric(names(tab)[which.min(tab)])
  n_new <- as.integer(max(tab) - min(tab))
  if (n_new <= 0) return(list(X = X, y = y))

  X_min <- X[y == min_lab, , drop = FALSE]
  if (nrow(X_min) <= k) k <- max(1, nrow(X_min) - 1)

  # Khoảng cách trong nội bộ lớp thiểu số; chặn đường chéo để không tự chọn mình
  D <- as.matrix(dist(X_min))
  diag(D) <- Inf

  synth <- matrix(0, nrow = n_new, ncol = ncol(X))
  for (i in seq_len(n_new)) {
    a <- sample.int(nrow(X_min), 1)
    neighbours <- order(D[a, ])[seq_len(k)] # k láng giềng gần nhất
    b <- neighbours[sample.int(k, 1)]         # chọn ngẫu nhiên 1 trong k
    gap <- runif(1)                            # điểm mới nằm trên đoạn a-b
    synth[i, ] <- X_min[a, ] + gap * (X_min[b, ] - X_min[a, ])
  }
  colnames(synth) <- colnames(X)
  list(X = rbind(X, synth), y = c(y, rep(min_lab, n_new)))
}
```

5.4 Hồi quy logistic có điều chuẩn ridge

Mô hình phân loại được sử dụng là hồi quy logistic có bổ sung phạt L_2 , còn gọi là điều chuẩn ridge. So với hồi quy logistic thông thường, hàm mục tiêu được cộng thêm một lượng phạt tỉ lệ với tổng bình phương các hệ số, với cường độ do tham số λ điều khiển. Hệ số chặn không bị phạt.

Vai trò của lượng phạt này là ổn định hoá nghiệm khi các biến giải thích còn tương quan với nhau. Trong trường hợp đó, mô hình không có điều chuẩn có thể gán cho hai biến tương quan hai hệ số rất lớn và ngược dấu, gần như triệt tiêu lẫn nhau; kết quả khớp dữ liệu vẫn tốt nhưng các hệ số dao động mạnh theo từng mẫu. Phạt L_2 khiến nghiệm co các hệ số này về phía nhau và về phía 0, đổi một chút độ chệch lấy mức giảm đáng kể của phương sai.

Giá trị λ càng lớn thì các hệ số càng bị co mạnh, kéo theo xác suất dự đoán tập trung quanh 0,5. Đây là điểm cần lưu ý vì nó ảnh hưởng trực tiếp tới bước lựa chọn ngưỡng phân loại ở Mục 9. Nghiệm được tìm bằng thuật toán IRLS (Iteratively Reweighted Least Squares), một quy trình lặp trong đó mỗi vòng lặp giải một bài toán bình phương tối thiểu có trọng số.

```
fit_logreg_ridge <- function(X, y, lambda = 0, max_iter = 50, tol = 1e-8) {
  Xd <- cbind(1, X) # thêm cột hệ số chặn
  p_dim <- ncol(Xd)
  beta <- rep(0, p_dim)

  P <- diag(lambda, p_dim)
  P[1, 1] <- 0 # không điều chuẩn hệ số chặn

  for (it in seq_len(max_iter)) {
    eta <- as.vector(Xd %*% beta)
    mu <- 1 / (1 + exp(-eta)) # xác suất dự đoán
    W <- mu * (1 - mu) # trọng số IRLS
    W[W < 1e-10] <- 1e-10 # chặn dưới để ma trận không suy biến
    z <- eta + (y - mu) / W # biến phản hồi hiệu chỉnh

    beta_new <- tryCatch(
      solve(t(Xd) %*% (Xd * W) + P, t(Xd) %*% (W * z)),
      error = function(e) beta
    )
    if (max(abs(beta_new - beta)) < tol) { beta <- beta_new; break }
    beta <- as.vector(beta_new)
  }
  as.vector(beta)
}

predict_proba <- function(beta, X) {
  1 / (1 + exp(-as.vector(cbind(1, X) %*% beta)))
}
```

5.5 Phòng ngừa rò rỉ dữ liệu

Đây là nội dung quan trọng nhất về mặt phương pháp luận. Quy trình huấn luyện gồm bốn bước, trong đó ba bước đầu đều ước lượng tham số từ dữ liệu:

1. điền khuyết, cần ước lượng trung vị của từng biến;
2. chuẩn hoá, cần ước lượng trung bình và độ lệch chuẩn của từng biến;
3. SMOTE, sinh mẫu mới dựa trên cấu trúc láng giềng của dữ liệu;
4. khớp mô hình hồi quy ridge.

Nếu ba bước đầu được thực hiện trên toàn bộ dữ liệu trước khi chia fold, một cách làm khá phổ biến vì tiện lợi, thì thông tin từ tập kiểm tra đã rò rỉ vào quá trình huấn luyện. Cụ thể, trung vị và độ lệch chuẩn dùng để biến đổi tập huấn luyện đã được tính với sự đóng góp của chính những quan sát sau đó dùng để đánh giá, khiến kết quả công bố lạc quan giả tạo. Với SMOTE, hậu quả còn nặng hơn: các mẫu tổng hợp nội suy từ quan sát nằm trong tập kiểm tra sẽ lọt vào tập huấn luyện, và mô hình gần như được biết trước đáp án.

Cách phòng ngừa đúng đắn là thực hiện lại toàn bộ bốn bước một cách độc lập bên trong mỗi fold, chỉ dựa trên phần dữ liệu huấn luyện của fold đó.

```
fit_pipeline <- function(X_tr, y_tr, lambda = 1, smote_k = 5, seed = 42) {
  # 1. Điền khuyết bằng median CỦA TẬP TRAIN
  med <- apply(X_tr, 2, median, na.rm = TRUE)
  for (j in seq_len(ncol(X_tr))) X_tr[is.na(X_tr[, j]), j] <- med[j]

  # 2. Chuẩn hoá theo mean/sd CỦA TẬP TRAIN
  mu <- colMeans(X_tr)
  sdv <- apply(X_tr, 2, sd)
  sdv[sdv == 0] <- 1
  X_tr_s <- scale(X_tr, center = mu, scale = sdv)

  # 3. SMOTE chỉ trên tập train
  res <- smote_oversample(X_tr_s, y_tr, k = smote_k, seed = seed)

  # 4. Khớp mô hình
  beta <- fit_logreg_ridge(res$X, res$y, lambda = lambda)

  list(beta = beta, median = med, mean = mu, sd = sdv)
}

# Áp dụng chuỗi biến đổi đã học từ train lên dữ liệu mới
pipeline_proba <- function(fit, X_new) {
  for (j in seq_len(ncol(X_new))) X_new[is.na(X_new[, j]), j] <- fit$median[j]
  X_s <- scale(X_new, center = fit$mean, scale = fit$sd)
  predict_proba(fit$beta, X_s)
}
```

Ngoài các hệ số hồi quy, quy trình còn lưu lại trung vị, trung bình và độ lệch chuẩn đã ước lượng từ tập huấn luyện. Đó chính là những tham số học được, và chúng phải được áp dụng nguyên vẹn khi biến đổi dữ liệu mới. Riêng SMOTE không xuất hiện ở bước dự đoán, bởi đây chỉ là kỹ thuật can thiệp vào tập huấn luyện và không bao giờ được áp lên dữ liệu cần dự đoán.

5.6 Bộ chỉ số đánh giá

```
compute_metrics <- function(y_true, y_pred) {
  tp <- sum(y_true == 1 & y_pred == 1)
  tn <- sum(y_true == 0 & y_pred == 0)
  fp <- sum(y_true == 0 & y_pred == 1)
  fn <- sum(y_true == 1 & y_pred == 0)

  recall1 <- if ((tp + fn) > 0) tp / (tp + fn) else NA # độ nhạy
  recall0 <- if ((tn + fp) > 0) tn / (tn + fp) else NA # độ đặc hiệu
  prec1 <- if ((tp + fp) > 0) tp / (tp + fp) else 0
  prec0 <- if ((tn + fn) > 0) tn / (tn + fn) else 0
  f1_1 <- if ((prec1 + recall1) > 0) 2*prec1*recall1/(prec1+recall1) else 0
  f1_0 <- if ((prec0 + recall0) > 0) 2*prec0*recall0/(prec0+recall0) else 0
}
```

```

c(accuracy = (tp + tn) / length(y_true),
  recall1 = recall1, precision1 = prec1,
  recall0 = recall0, precision0 = prec0,
  macro_f1 = (f1_1 + f1_0) / 2)
}

print_confusion <- function(y_true, y_pred, title = "MA TRAN NHAM LAN") {
  tp <- sum(y_true == 1 & y_pred == 1); tn <- sum(y_true == 0 & y_pred == 0)
  fp <- sum(y_true == 0 & y_pred == 1); fn <- sum(y_true == 1 & y_pred == 0)

  cat("\n", title, "\n", sep = "")
  print(matrix(c(tn, fp, fn, tp), nrow = 2, byrow = TRUE,
    dimnames = list(`Thực tế` = c("Không bệnh", "Có bệnh"),
                     `Dự đoán` = c("Không bệnh", "Có bệnh"))))
  cat(sprintf("  TN = %3d   không bệnh, dự đoán đúng\n", tn))
  cat(sprintf("  FP = %3d   không bệnh, dự đoán nhầm là có bệnh\n", fp))
  cat(sprintf("  FN = %3d   có bệnh, bị bỏ sót\n", fn))
  cat(sprintf("  TP = %3d   có bệnh, phát hiện được\n", tp))
  invisible(compute_metrics(y_true, y_pred))
}

```

Macro F1, tức trung bình cộng chỉ số F1 của hai lớp, được chọn làm chỉ số tổng hợp vì nó gán trọng số bằng nhau cho hai lớp bất kể chênh lệch cỡ, phù hợp với đặc thù của bài toán mất cân bằng.

6 Ước lượng khoảng tin cậy bằng Bootstrap

Recall của lớp có bệnh chỉ là một con số tính từ hơn một trăm bệnh nhân trong tập kiểm tra. Với một nhóm bệnh nhân khác, con số đó sẽ khác đi. Bootstrap được dùng để ước lượng chính mức dao động ấy.

Về thiết kế, mô hình được huấn luyện một lần và dự đoán một lần, sau đó kết quả dự đoán được giữ cố định trong suốt vòng lặp. Mỗi lần lặp chỉ rút lại tập kiểm tra theo cách có hoàn lại. Khoảng tin cậy thu được vì thế phản ánh độ bất định đến từ việc lấy mẫu bệnh nhân kiểm tra, chứ không bao gồm độ bất định do thay đổi tập huấn luyện. Đây là một giới hạn được bàn thêm ở phần kết luận.

```

sp <- stratified_split(y_all, test_frac = 0.2, seed = 42)
X_train <- X_all[sp$train, , drop = FALSE]; y_train <- y_all[sp$train]
X_test  <- X_all[sp$test, , drop = FALSE]; y_test  <- y_all[sp$test]

cat("Train:", nrow(X_train), "mẫu | Test:", nrow(X_test), "mẫu\n")

## Train: 467 mẫu | Test: 116 mẫu

# Huấn Luyện MỘT lần, dự đoán MỘT lần
fit_80 <- fit_pipeline(X_train, y_train, lambda = 1, smote_k = 5)
proba_test <- pipeline_proba(fit_80, X_test)
pred_test <- as.integer(proba_test >= 0.5)

recall_point <- compute_metrics(y_test, pred_test)["recall1"]

```

```

CONF <- 0.90
B <- 10000
n_test <- length(y_test)

recalls <- numeric(0)

```

```

for (b in seq_len(B)) {
  # Rút CỎ HOÀN LẠI, cùng cỡ mẫu
  idx <- sample.int(n_test, n_test, replace = TRUE)
  if (sum(y_test[idx] == 1) == 0) next # không có ca bệnh -> recall vô nghĩa
  recalls <- c(recalls,
               compute_metrics(y_test[idx], pred_test[idx])["recall1"])
}

# Khoảng tin cậy percentile: cắt (1-CONF)/2 ở MỖI đầu
tail_pct <- (1 - CONF) / 2
ci <- quantile(recalls, c(tail_pct, 1 - tail_pct))

cat(sprintf("Recall lớp có bệnh   = %.3f\n", recall_point))

## Recall lớp có bệnh   = 0.614

cat(sprintf("Khoảng tin cậy %.0f%%   = [%.3f, %.3f]\n",
            CONF * 100, ci[1], ci[2]))

## Khoảng tin cậy 90%   = [0.526, 0.700]

cat(sprintf("Bề rộng khoảng       = %.3f\n", ci[2] - ci[1]))

## Bề rộng khoảng       = 0.174

cat(sprintf("Sai số chuẩn         = %.3f\n", sd(recalls)))

## Sai số chuẩn         = 0.053

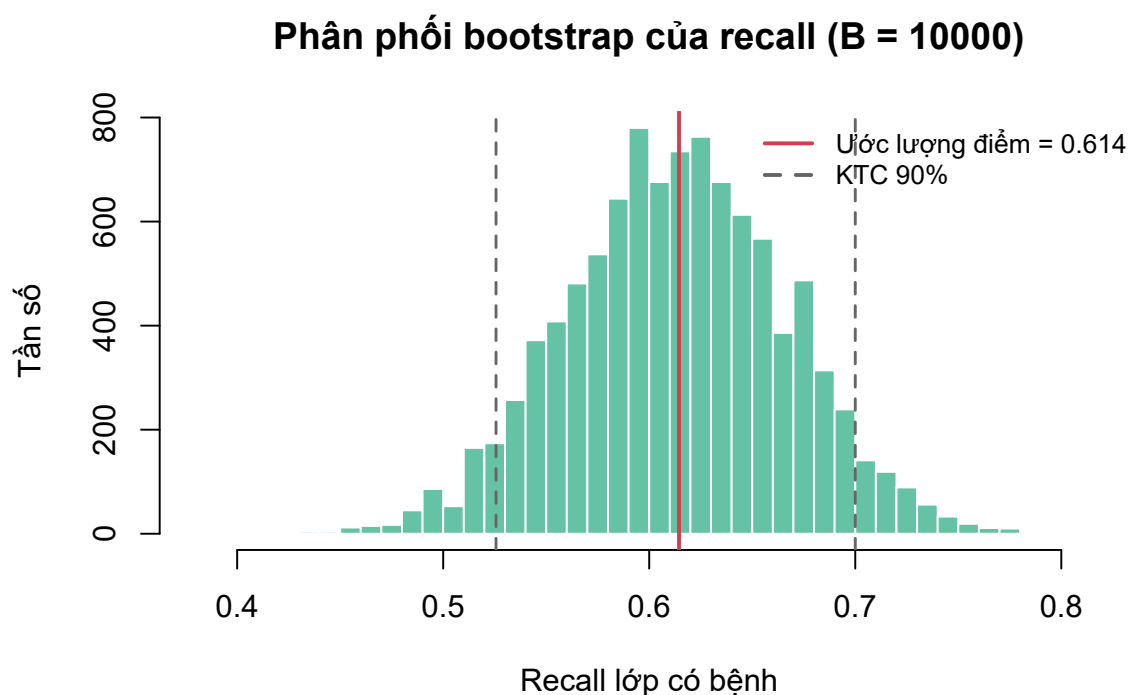
hist(recalls, breaks = 40, col = "#66c2a5", border = "white",
     main = sprintf("Phân phối bootstrap của recall (B = %d)",
                    length(recalls)),
     xlab = "Recall lớp có bệnh", ylab = "Tần số")
abline(v = recall_point, col = "#d53e4f", lwd = 2)
abline(v = ci, col = "gray40", lty = 2, lwd = 1.5)
legend("topright", bty = "n", cex = 0.85,
      legend = c(sprintf("Ước lượng điểm = %.3f", recall_point),
                  sprintf("KTC %.0f%%", CONF * 100)),
      col = c("#d53e4f", "gray40"), lty = c(1, 2), lwd = 2)

```

Với ngưỡng mặc định 0,5, recall của lớp có bệnh đạt 0,614 và khoảng tin cậy 90% là [0,526; 0,700], tức bề rộng 0,174 hay gần 17 điểm phần trăm. Con số này rất đáng lưu ý: nếu chỉ công bố giá trị recall bằng 0,61, báo cáo sẽ bỏ qua một mức bất định rất lớn, mà nguyên nhân hoàn toàn nằm ở cỡ mẫu kiểm tra chỉ gồm 116 quan sát, trong đó có 83 ca bệnh. Đây chính là loại thông tin mà một ước lượng điểm đơn lẻ không thể truyền tải. Hình 5 cho thấy phân phối bootstrap tương đối cân đối, phù hợp với việc sử dụng khoảng tin cậy phân vị.

7 Kiểm định chéo phân tầng

Phân trên đánh giá mô hình trên một lần chia 80:20 duy nhất. Kiểm định chéo luân phiên vai trò của cả 5 phân, nhờ đó mọi quan sát đều được dự đoán đúng một lần bởi một mô hình chưa từng sử dụng nó trong huấn luyện.



Hình 5: Phân phối bootstrap của recall lớp có bệnh

Giá trị $k = 5$ được chọn với lý do cụ thể: mỗi fold kiểm tra đúng 20% dữ liệu, trùng với tỉ lệ chia 80:20 ở phần trước. Nhờ vậy hai cách đánh giá cùng áp dụng cho mô hình huấn luyện trên khoảng 466 quan sát và có thể so sánh trực tiếp với nhau.

```
K <- 5
folds <- stratified_folds(y_all, k = K, seed = 42)

cv_results <- data.frame()
for (i in seq_len(K)) {
  te <- which(folds == i)
  tr <- which(folds != i)

  fit_i <- fit_pipeline(X_all[tr, , drop = FALSE], y_all[tr],
                        lambda = 1, smote_k = 5)
  pred_i <- as.integer(pipeline_proba(fit_i,
                                     X_all[te, , drop = FALSE]) >= 0.5)

  m <- compute_metrics(y_all[te], pred_i)
  cv_results <- rbind(cv_results, data.frame(
    Fold = i, n_train = length(tr), n_test = length(te),
    Accuracy = m["accuracy"], Recall = m["recall1"],
    Macro_F1 = m["macro_f1"]
  ))
}
rownames(cv_results) <- NULL
print(round(cv_results, 4), row.names = FALSE)
```

##	Fold	n_train	n_test	Accuracy	Recall	Macro_F1
##	1	465	118	0.6186	0.5238	0.6124
##	2	466	117	0.6923	0.6386	0.6776
##	3	467	116	0.6466	0.6145	0.6263
##	4	467	116	0.6466	0.6265	0.6230
##	5	467	116	0.6897	0.6265	0.6758

```
for (m in c("Accuracy", "Recall", "Macro_F1")) {
  cat(sprintf(" %-9s = %.3f +/- %.3f\n",
             m, mean(cv_results[[m]]), sd(cv_results[[m]])))
}

## Accuracy = 0.659 +/- 0.032
## Recall   = 0.606 +/- 0.047
## Macro_F1 = 0.643 +/- 0.031
```

Kết quả được báo cáo dưới dạng trung bình kèm độ lệch chuẩn giữa các fold, thay vì một con số đơn lẻ. Độ lệch chuẩn ở đây mang ý nghĩa riêng, cho biết kết quả dao động bao nhiêu khi chuyển sang một cách chia dữ liệu khác. Recall dao động mạnh nhất, đạt $0,606 \pm 0,047$ và trải từ 0,524 ở fold thứ nhất tới 0,639 ở fold thứ hai, trong khi macro F1 ổn định hơn với $0,643 \pm 0,031$.

Một điểm đáng chú ý là recall trung bình qua 5 fold, bằng 0,606, rất gần với recall đo được trên một lần chia 80:20 ở phần trước, bằng 0,614, và cả hai đều nằm gọn trong khoảng tin cậy bootstrap $[0,526; 0,700]$. Hai phương pháp resampling độc lập cho kết quả nhất quán, cho thấy ước lượng không phải sản phẩm của một lần chia dữ liệu thuận lợi.

8 Tinh chỉnh siêu tham số

Từ đây trở đi, quy trình tuân thủ nghiêm ngặt nguyên tắc phân vai: kiểm định chéo chạy trên tập huấn luyện để tinh chỉnh, còn tập kiểm tra được giữ nguyên vẹn và chỉ sử dụng ở phần cuối cùng.

Hai siêu tham số được dò trên lưới là cường độ điều chuẩn λ của ridge và số láng giềng k mà SMOTE dùng để nội suy, tạo thành 15 cấu hình. Mỗi cấu hình được đánh giá bằng kiểm định chéo 5 fold trên tập huấn luyện, tương ứng 75 lần huấn luyện mô hình. Tiêu chí chấm điểm là macro F1 chứ không phải recall của lớp có bệnh. Lý do là việc tối đa hoá recall lớp có bệnh có một nghiệm tầm thường, đó là dự đoán tất cả quan sát đều mắc bệnh, cho recall bằng 1 nhưng không có giá trị sử dụng.

```
param_grid <- expand.grid(
  lambda = c(0.01, 0.1, 1, 10, 100), # cường độ điều chuẩn ridge
  smote_k = c(3, 5, 7)               # số láng giềng SMOTE
)

folds_inner <- stratified_folds(y_train, k = 5, seed = 42)

grid_mean <- grid_sd <- numeric(nrow(param_grid))
for (g in seq_len(nrow(param_grid))) {
  scores <- numeric(5)
  for (i in seq_len(5)) {
    te <- which(folds_inner == i); tr <- which(folds_inner != i)
    fit_g <- fit_pipeline(X_train[tr, , drop = FALSE], y_train[tr],
                        lambda = param_grid$lambda[g],
                        smote_k = param_grid$smote_k[g])
    pred_g <- as.integer(pipeline_proba(
      fit_g, X_train[te, , drop = FALSE]) >= 0.5)
    scores[i] <- compute_metrics(y_train[te], pred_g)["macro_f1"]
  }
}
```

```

grid_mean[g] <- mean(scores)
grid_sd[g] <- sd(scores) # dao động giữa 5 fold, đo mức ổn định
}

param_grid$macro_f1 <- grid_mean
param_grid$sd_fold <- grid_sd

print(head(param_grid[order(-param_grid$macro_f1), ], 5), row.names = FALSE)

## lambda smote_k macro_f1 sd_fold
## 100 7 0.6440162 0.01687905
## 100 5 0.6408597 0.01098893
## 100 3 0.6407688 0.01654468
## 1 7 0.6406804 0.03143136
## 10 7 0.6398884 0.02943170

```

Cấu hình đạt điểm cao nhất là $\lambda = 100$ với $k_{SMOTE} = 7$, cho macro F1 bằng 0,6440. Tuy nhiên chênh lệch giữa năm cấu hình dẫn đầu chỉ khoảng 0,004, trong khi sai số chuẩn của trung bình 5 fold là 0,0075. Mức chênh lệch giữa các cấu hình như vậy còn nhỏ hơn sai số của chính phép đo, nên thứ hạng này không đủ vững để khẳng định cấu hình dẫn đầu thực sự tốt hơn những cấu hình còn lại.

Vì lý do đó, mọi cấu hình nằm trong phạm vi một sai số chuẩn quanh điểm cao nhất được xem là tương đương nhau, và có 14 trên 15 cấu hình thoả điều kiện này. Trong nhóm tương đương, cấu hình có λ nhỏ nhất được chọn, căn cứ vào đặc điểm đã nêu ở Mục 5.4: giá trị λ lớn nén xác suất dự đoán quanh 0,5 và làm bước lựa chọn ngưỡng ở phần sau kém hiệu quả.

```

i_top <- which.max(param_grid$macro_f1)
se_top <- param_grid$sd_fold[i_top] / sqrt(5) # sai số chuẩn của trung bình

# Nhóm cấu hình tương đương: cách điểm cao nhất không quá một sai số chuẩn
tied <- which(param_grid$macro_f1 >= param_grid$macro_f1[i_top] - se_top)

# Trong nhóm đó, ưu tiên lambda nhỏ nhất
cand <- tied[param_grid$lambda[tied] == min(param_grid$lambda[tied])]
best_idx <- cand[which.max(param_grid$macro_f1[cand])]
best <- param_grid[best_idx, ]

cat(sprintf("\nSai số chuẩn của trung bình 5 fold: %.4f\n", se_top))

##
## Sai số chuẩn của trung bình 5 fold: 0.0075

cat(sprintf("Số cấu hình tương đương: %d/%d\n",
            length(tied), nrow(param_grid)))

## Số cấu hình tương đương: 14/15

cat(sprintf("Cấu hình được chọn: lambda = %g, smote_k = %g (macro F1 = %.4f)\n",
            best$lambda, best$smote_k, best$macro_f1))

## Cấu hình được chọn: lambda = 0.01, smote_k = 3 (macro F1 = 0.6397)

cat(sprintf("Đã thử %d cấu hình x 5 fold = %d lần khớp mô hình\n",
            nrow(param_grid), nrow(param_grid) * 5))

## Đã thử 15 cấu hình x 5 fold = 75 lần khớp mô hình

```

9 Lựa chọn ngưỡng phân loại

Ngưỡng mặc định 0,5 tối ưu cho độ chính xác tổng thể và không ưu tiên lớp nào. Bài toán chẩn đoán bệnh lại khác. Bỏ sót một ca bệnh gây hậu quả nặng hơn nhiều so với báo động nhầm một người khoẻ mạnh, bởi báo động nhầm chỉ dẫn tới một xét nghiệm bổ sung, trong khi bỏ sót có thể khiến bệnh tiến triển mà không được phát hiện. Vì vậy ngưỡng được hạ xuống để tăng recall của lớp có bệnh.

Tiêu chí lựa chọn là **tối thiểu hoá tổng số ca phân loại sai**, tức tổng của số ca bệnh bị bỏ sót và số người khoẻ bị báo nhầm. Cần nhấn mạnh rằng đây là tiêu chí đếm trực tiếp trên số bệnh nhân, khác với các chỉ số tỉ lệ như recall hay macro F1. Sự phân biệt này quan trọng vì hai lớp có cỡ rất khác nhau: một điểm phần trăm recall của lớp có bệnh tương ứng với nhiều bệnh nhân hơn hẳn so với một điểm phần trăm của lớp không bệnh. Tiêu chí đếm vì thế tự động đặt trọng số cao hơn cho lớp đông, ở đây chính là lớp có bệnh.

Ngưỡng bắt buộc phải được xác định trên tập huấn luyện. Nếu dò trên tập kiểm tra thì tập này đã tham gia vào một quyết định của quá trình xây dựng mô hình, và con số công bố cuối cùng sẽ lạc quan giả tạo. Ở đây, ngưỡng được xác định thông qua dự đoán *out-of-fold*, trong đó mỗi quan sát của tập huấn luyện được chấm điểm bởi mô hình chưa từng sử dụng nó.

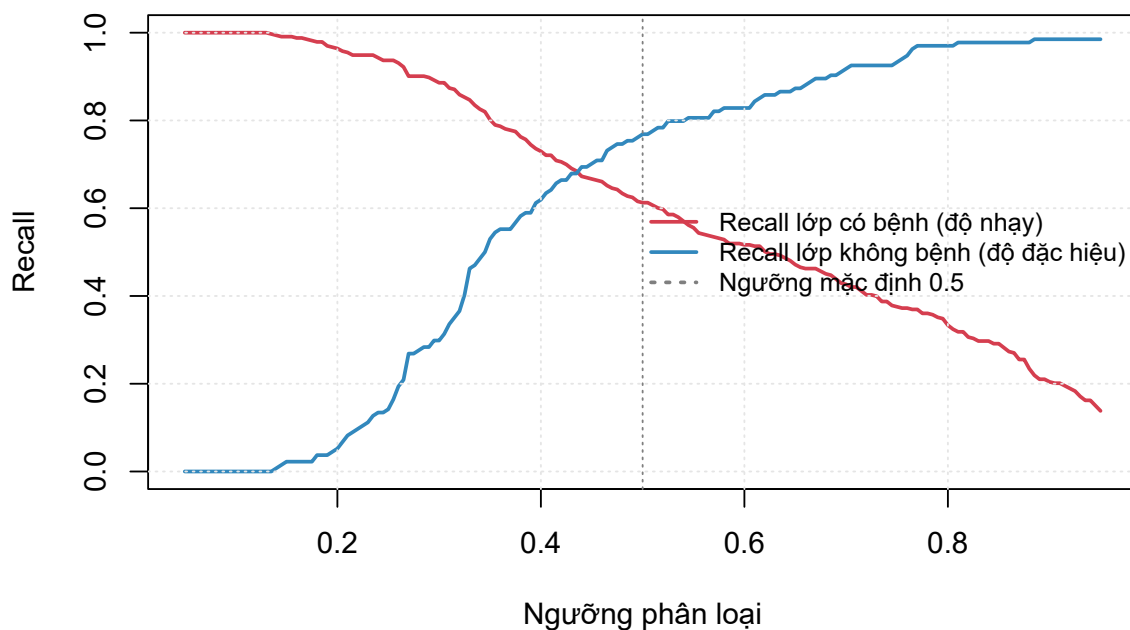
```
oof_proba <- numeric(length(y_train))
for (i in seq_len(5)) {
  te <- which(folds_inner == i); tr <- which(folds_inner != i)
  fit_o <- fit_pipeline(X_train[tr, , drop = FALSE], y_train[tr],
                       lambda = best$lambda, smote_k = best$smote_k)
  oof_proba[te] <- pipeline_proba(fit_o, X_train[te, , drop = FALSE])
}

grid_thr <- seq(0.05, 0.95, by = 0.005)
rec1 <- rec0 <- n_fn <- n_fp <- numeric(length(grid_thr))
for (i in seq_along(grid_thr)) {
  pred_t <- as.integer(oof_proba >= grid_thr[i])
  m <- compute_metrics(y_train, pred_t)
  rec1[i] <- m["recall1"]; rec0[i] <- m["recall0"]
  n_fn[i] <- sum(y_train == 1 & pred_t == 0) # ca bệnh bị bỏ sót
  n_fp[i] <- sum(y_train == 0 & pred_t == 1) # người khoẻ bị báo nhầm
}
```

```
plot(grid_thr, rec1, type = "l", col = "#d53e4f", lwd = 2, ylim = c(0, 1),
     xlab = "Ngưỡng phân loại", ylab = "Recall",
     main = "Recall hai lớp theo ngưỡng (out-of-fold trên tập train)")
lines(grid_thr, rec0, col = "#3288bd", lwd = 2)
abline(v = 0.5, col = "gray50", lty = 3)
grid(col = "gray90")
legend("right", bty = "n", cex = 0.85, lwd = 2,
      legend = c("Recall lớp có bệnh (độ nhạy)",
                  "Recall lớp không bệnh (độ đặc hiệu)",
                  "Ngưỡng mặc định 0.5"),
      col = c("#d53e4f", "#3288bd", "gray50"), lty = c(1, 1, 3))
```

Hình 6 thể hiện rõ bản chất đánh đổi giữa hai chỉ số: hai đường đi ngược chiều nhau, và mọi mức tăng của recall lớp có bệnh đều phải trả bằng mức giảm tương ứng của recall lớp không bệnh. Bảng tra dưới đây lượng hoá cái giá đó.

Recall hai lớp theo ngưỡng (out-of-fold trên tập train)



Hình 6: Đánh đổi giữa độ nhạy và độ đặc hiệu khi thay đổi ngưỡng phân loại

```
sel <- which(round(grid_thr, 3) %in% c(0.50, 0.45, 0.40, 0.375, 0.35,
                                     0.33, 0.30, 0.25))

print(data.frame(
  nguong = grid_thr[sel], FN = n_fn[sel], FP = n_fp[sel],
  tong_sai = n_fn[sel] + n_fp[sel],
  recall_lop1 = round(rec1[sel], 3), recall_lop0 = round(rec0[sel], 3)
), row.names = FALSE)

## nguong  FN  FP tong_sai recall_lop1 recall_lop0
## 0.250  21 115      136      0.937      0.142
## 0.300  38  94      132      0.886      0.299
## 0.330  51  72      123      0.847      0.463
## 0.350  66  63      129      0.802      0.530
## 0.375  75  58      133      0.775      0.567
## 0.400  90  51      141      0.730      0.619
## 0.450 111  40      151      0.667      0.701
## 0.500 129  31      160      0.613      0.769

threshold <- grid_thr[which.min(n_fn + n_fp)]
i_thr <- which(grid_thr == threshold)

cat(sprintf("\nNgưỡng mặc định : 0.500\n"))

##
## Ngưỡng mặc định : 0.500

cat(sprintf("Ngưỡng đã chọn : %.3f\n", threshold))

## Ngưỡng đã chọn : 0.330

cat(sprintf(" Trên out-of-fold: FN = %d, FP = %d, tổng = %d\n",
  n_fn[i_thr], n_fp[i_thr], n_fn[i_thr] + n_fp[i_thr]))

## Trên out-of-fold: FN = 51, FP = 72, tổng = 123
```

Bảng tra cho thấy tổng số ca sai đạt cực tiểu tại ngưỡng 0,330, với 51 ca bệnh bị bỏ sót và 72 người khỏe bị báo nhầm trên tổng 123 ca sai. Ở ngưỡng mặc định 0,500, tổng này là 160 ca, trong đó riêng số ca bệnh bị bỏ sót đã là 129. Ngưỡng 0,330 do đó vừa giảm tổng sai sót, vừa đảo ngược tương quan giữa hai loại lỗi: số ca bệnh bỏ sót trở nên ít hơn số người khỏe bị báo nhầm.

Hai đầu bảng minh họa giới hạn của việc điều chỉnh ngưỡng. Hạ tiếp xuống 0,250 thì số ca bệnh bỏ sót giảm còn 21, nhưng số người khỏe bị báo nhầm tăng lên 115 trên tổng 134, tức mô hình xếp gần như toàn bộ người khỏe vào nhóm có bệnh và mất giá trị sàng lọc. Ngược lại, nâng lên 0,500 thì độ đặc hiệu đạt 0,769 nhưng phải trả bằng 129 ca bệnh bị bỏ sót.

10 Đánh giá trên tập kiểm tra

Đây là lần đầu tiên tập kiểm tra được sử dụng kể từ đầu quy trình. Mô hình được khớp lại trên toàn bộ tập huấn luyện với siêu tham số đã chọn, sau đó áp ngưỡng 0,330.

```
fit_best <- fit_pipeline(X_train, y_train,
                        lambda = best$lambda, smote_k = best$smote_k)
proba_final <- pipeline_proba(fit_best, X_test)
pred_final <- as.integer(proba_final >= threshold)
pred_05 <- as.integer(proba_final >= 0.5)

m_final <- print_confusion(y_test, pred_final,
                          sprintf("MA TRAN NHAM LAN (nguong %.3f)",
                                  threshold))

##
## MA TRAN NHAM LAN (nguong 0.330)
##
##          Dự đoán
## Thực tế   Không bệnh   Có bệnh
## Không bệnh      14       19
## Có bệnh         15       68
## TN = 14   không bệnh, dự đoán đúng
## FP = 19   không bệnh, dự đoán nhầm là có bệnh
## FN = 15   có bệnh, bị bỏ sót
## TP = 68   có bệnh, phát hiện được

cat("\nCác chỉ số trên tập kiểm tra:\n")

##
## Các chỉ số trên tập kiểm tra:

print(round(m_final, 3))

## accuracy   recall1 precision1   recall0 precision0   macro_f1
##      0.707      0.819      0.782      0.424      0.483      0.626

cmp <- rbind(
  `0.500 (mặc định)` = compute_metrics(y_test, pred_05),
  `đã chỉnh`        = compute_metrics(y_test, pred_final)
)
print(round(cmp, 3))
```

```
##          accuracy recall1 precision1 recall0 precision0
## 0.500 (mặc định)    0.681    0.614      0.911    0.848      0.467
## đã chỉnh          0.707    0.819      0.782    0.424      0.483
##          macro_f1
## 0.500 (mặc định)    0.668
## đã chỉnh          0.626
```

Việc hạ ngưỡng từ 0,500 xuống 0,330 đạt được mục tiêu đặt ra. Số ca bệnh bị bỏ sót giảm từ 32 xuống 15, tức phát hiện thêm 17 bệnh nhân, trong khi số người khỏe bị báo nhầm tăng từ 5 lên 19. Quan hệ giữa hai loại lỗi đảo chiều: ở ngưỡng mặc định, số ca bỏ sót gấp hơn sáu lần số báo động nhầm; ở ngưỡng đã chọn, số ca bỏ sót đã ít hơn số báo động nhầm. Recall của lớp có bệnh tăng từ 0,614 lên 0,819, và độ chính xác tổng thể cũng nhích từ 0,681 lên 0,707.

Cái giá phải trả nằm ở lớp không bệnh. Độ đặc hiệu giảm mạnh từ 0,848 xuống 0,424, nghĩa là hơn một nửa số người khỏe mạnh bị xếp nhầm vào nhóm có bệnh. Precision của lớp có bệnh giảm từ 0,911 xuống 0,782, và macro F1 giảm từ 0,668 xuống 0,626. Đây là đánh đổi có chủ đích: tiêu chí lựa chọn ngưỡng đặt trọng tâm vào việc giảm số ca bệnh bị bỏ sót, nên tất yếu phải chấp nhận nhiều báo động nhầm hơn.

Mức đánh đổi này cũng được tái lập khá sát giữa hai tập dữ liệu. Trên dự đoán out-of-fold của tập huấn luyện, ngưỡng 0,330 cho recall 0,847 và độ đặc hiệu 0,463; trên tập kiểm tra, hai giá trị tương ứng là 0,819 và 0,424. Sự tương đồng cho thấy ngưỡng chọn được không chỉ phù hợp riêng với dữ liệu dùng để dò nó.

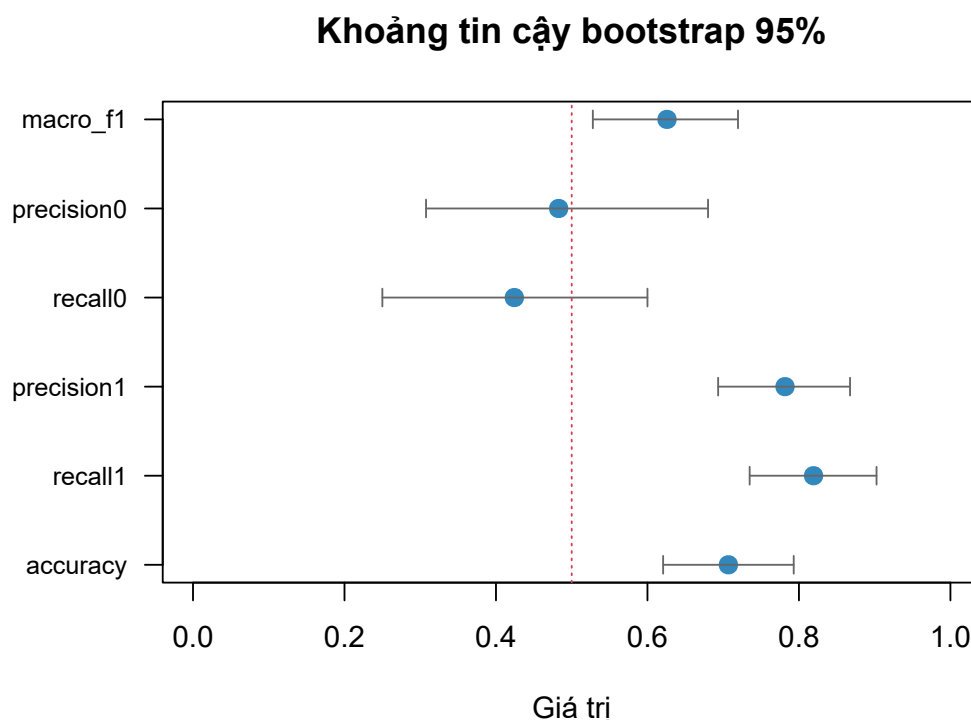
10.1 Khoảng tin cậy bootstrap cho toàn bộ chỉ số

```
CONF2 <- 0.95
B2 <- 2000
boot_mat <- matrix(NA, nrow = B2, ncol = length(m_final),
  dimnames = list(NULL, names(m_final)))

for (b in seq_len(B2)) {
  idx <- sample.int(n_test, n_test, replace = TRUE)
  if (length(unique(y_test[idx])) < 2) next # mẫu suy biến, bỏ qua
  boot_mat[b, ] <- compute_metrics(y_test[idx], pred_final[idx])
}
boot_mat <- boot_mat[complete.cases(boot_mat), , drop = FALSE]

tail2 <- (1 - CONF2) / 2
ci_tbl <- data.frame(
  diem_uoc_luong = m_final,
  ci_duoi = apply(boot_mat, 2, quantile, probs = tail2),
  ci_tren = apply(boot_mat, 2, quantile, probs = 1 - tail2),
  sai_so_chuan = apply(boot_mat, 2, sd)
)
print(round(ci_tbl, 4))

##          diem_uoc_luong ci_duoi ci_tren sai_so_chuan
## accuracy          0.7069  0.6207  0.7931    0.0427
## recall1           0.8193  0.7349  0.9024    0.0432
## precision1        0.7816  0.6933  0.8675    0.0448
## recall0           0.4242  0.2500  0.6000    0.0872
## precision0        0.4828  0.3077  0.6800    0.0941
## macro_f1          0.6258  0.5279  0.7195    0.0492
```



Hình 7: Khoảng tin cậy bootstrap 95% cho sáu chỉ số trên tập kiểm tra

```
par(mar = c(5, 9, 4, 2))
n_m <- nrow(ci_tbl)
plot(ci_tbl$diem_uoc_luong, seq_len(n_m), pch = 19, col = "#3288bd",
     cex = 1.3, xlim = c(0, 1), yaxt = "n", ylab = "", xlab = "Giá trị",
     main = sprintf("Khoảng tin cậy bootstrap %.0f%%", CONF2 * 100))
arrows(ci_tbl$ci_duoi, seq_len(n_m), ci_tbl$ci_tren, seq_len(n_m),
       length = 0.05, angle = 90, code = 3, col = "gray40")
axis(2, seq_len(n_m), rownames(ci_tbl), las = 2, cex.axis = 0.85)
abline(v = 0.5, col = "#d53e4f", lty = 3)
```

```
par(mar = c(5, 4, 4, 2) + 0.1)
```

Kết quả chính của báo cáo là recall lớp có bệnh đạt 0,819 với khoảng tin cậy 95% là [0,735; 0,902].

Hình 7 cho thấy độ rộng khoảng tin cậy khác nhau đáng kể giữa các chỉ số, và sự khác biệt đó phản ánh trực tiếp cỡ mẫu dùng để tính từng chỉ số. Recall của lớp có bệnh có khoảng hẹp nhất, với sai số chuẩn 0,043, vì được tính trên toàn bộ 83 ca bệnh. Ngược lại, hai chỉ số của lớp không bệnh có khoảng rộng nhất: độ đặc hiệu bằng [0,250; 0,600] và precision bằng [0,308; 0,680], đều với sai số chuẩn gần 0,09. Nguyên nhân là chúng chỉ được tính trên 33 người khỏe và 29 dự đoán âm tính. Nói cách khác, các ước lượng 0,424 và 0,483 cho hai chỉ số này mang rất ít thông tin xác định, và đó là điều mà một ước lượng điểm đơn lẻ hoàn toàn không thể hiện được.

11 Kết luận

Báo cáo đã xây dựng một quy trình phân loại hoàn chỉnh trên bộ dữ liệu ILPD và, quan trọng hơn, minh họa sự phân công vai trò rõ ràng giữa hai kỹ thuật resampling.

Vai trò của kiểm định chéo trong lựa chọn mô hình. Kiểm định chéo phân tầng 5 phần được sử dụng ở ba vị trí: đánh giá mô hình cơ sở với macro F1 đạt $0,643 \pm 0,031$, dò 15 cấu hình siêu tham số qua 75 lần khớp mô hình, và sinh dự đoán out-of-fold phục vụ lựa chọn ngưỡng phân loại. Ngoài giá trị trung bình, kiểm định chéo còn cung cấp độ lệch chuẩn giữa các fold. Chính đại lượng này cho thấy 14 trên 15 cấu hình siêu tham số nằm trong phạm vi một sai số chuẩn của nhau, nghĩa là lưới tham số về cơ bản không phân biệt được chúng, và việc chọn cấu hình có điểm cao nhất sẽ là chọn theo nhiều. Một lần chia train/test đơn lẻ không thể cung cấp thông tin này.

Vai trò của Bootstrap trong định lượng độ tin cậy. Mô hình cuối đạt recall 0,819 trên tập kiểm tra, nhưng khoảng tin cậy 95% trải từ 0,735 tới 0,902. Bề rộng gần 17 điểm phần trăm này là thông tin thiết yếu, bởi nó cho thấy với cỡ mẫu hiện có, không thể phân biệt được mô hình này với một mô hình có recall thực sự bằng 0,74 hay 0,90. Mọi so sánh với các nghiên cứu khác trên cùng bộ dữ liệu đều cần được diễn giải trong bối cảnh đó.

Về mô hình và ngưỡng phân loại. Cấu hình cuối cùng là hồi quy logistic có điều chuẩn ridge với $\lambda = 0,01$, kết hợp SMOTE với $k = 3$ láng giềng và ngưỡng phân loại 0,330, sử dụng 7 biến còn lại sau khi loại Direct_Bilirubin do đa cộng tuyến cùng Gender và Total_Protiens do không đạt ngưỡng ý nghĩa sau hiệu chỉnh FDR. Nhóm biến quan trọng nhất là các men gan và bilirubin, với kích thước ảnh hưởng rank-biserial trong khoảng 0,35 đến 0,39.

Ngưỡng 0,330 được chọn theo tiêu chí tối thiểu hoá tổng số ca phân loại sai, thay vì tối ưu một chỉ số tỉ lệ. Lựa chọn này phản ánh ưu tiên lâm sàng của bài toán sàng lọc, khi bỏ sót một ca bệnh gây hậu quả nặng hơn một báo động nhầm. Kết quả trên tập kiểm tra là 15 ca bệnh bị bỏ sót so với 19 người khỏe bị báo nhầm, tức lỗi tổn kém hơn đã trở thành lỗi ít gặp hơn. Đổi lại, độ đặc hiệu chỉ còn 0,424 và macro F1 giảm xuống 0,626; đây là cái giá trực tiếp của ưu tiên nói trên, không phải khuyết điểm của mô hình.

11.1 Hạn chế

1. **Hiệu năng tuyệt đối còn thấp.** Recall 0,819 vẫn đồng nghĩa với việc bỏ sót 15 trên 83 bệnh nhân trong tập kiểm tra. Ở chiều ngược lại, độ đặc hiệu chỉ đạt 0,424 nên 19 trên 33 người khỏe mạnh bị báo nhầm, kéo theo chi phí xét nghiệm bổ sung đáng kể nếu triển khai thực tế. Precision của lớp không bệnh đạt 0,483, nghĩa là trong số những người mô hình kết luận

không mắc bệnh, hơn một nửa thực tế có bệnh.

2. **Khoảng tin cậy chỉ phản ánh một nguồn bất định.** Do mô hình được giữ cố định trong suốt vòng lặp và chỉ tập kiểm tra được lấy mẫu lại, khoảng thu được phản ánh biến thiên do lấy mẫu bệnh nhân kiểm tra, chứ không tính tới biến thiên do thay đổi tập huấn luyện. Độ bất định thực tế vì vậy rộng hơn con số đã công bố.
3. **Cỡ mẫu hạn chế.** Bộ dữ liệu chỉ có 583 quan sát, trong đó tập kiểm tra gồm 116 quan sát. Đây là nguyên nhân trực tiếp của các khoảng tin cậy rộng và của việc nhiều so sánh không đạt mức phân biệt thống kê.
4. **Tính đại diện.** Dữ liệu được thu thập tại một vùng địa lý duy nhất là đông bắc bang Andhra Pradesh, Ấn Độ, nên kết quả chưa chắc tổng quát hoá được sang quần thể khác.
5. **Đa cộng tuyến chưa được loại bỏ hoàn toàn.** Hai biến Albumin và Albumin_and_Globulin_Ratio vẫn liên quan chặt về mặt định nghĩa. Điều chuẩn ridge làm giảm ảnh hưởng của vấn đề này nhưng không loại bỏ được nó, và hệ quả là các hệ số riêng lẻ của mô hình không nên được diễn giải như mức độ quan trọng của từng biến.

11.2 Hướng mở rộng

Ba hướng mở rộng được đề xuất theo thứ tự ưu tiên. Thứ nhất, huấn luyện lại mô hình trong từng lần lặp bootstrap để khoảng tin cậy bao gồm cả biến thiên do quá trình huấn luyện, đổi lại chi phí tính toán cao hơn nhiều. Thứ hai, lặp lại toàn bộ quy trình kiểm định chéo với nhiều cách chia khác nhau nhằm giảm phương sai của ước lượng và kiểm tra xem kết luận về sự tương đương giữa 14 trên 15 cấu hình có ổn định hay không. Thứ ba, mở rộng sang các họ mô hình phi tuyến như random forest hay gradient boosting, vốn có khả năng nắm bắt tương tác giữa các chỉ số men gan mà mô hình tuyến tính bỏ qua. Khi đó, chính khung kiểm định chéo đã thiết lập trong báo cáo này sẽ là công cụ so sánh khách quan giữa các họ mô hình.

12 Tài liệu tham khảo

1. Benjamini, Y., & Hochberg, Y. (1995). Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing. *Journal of the Royal Statistical Society, Series B*, 57(1), 289–300.

2. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
3. Efron, B. (1979). Bootstrap Methods: Another Look at the Jackknife. *The Annals of Statistics*, 7(1), 1–26.
4. Efron, B., & Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall.
5. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer.
6. Hoerl, A. E., & Kennard, R. W. (1970). Ridge Regression: Biased Estimation for Nonorthogonal Problems. *Technometrics*, 12(1), 55–67.
7. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning*. Springer.
8. Kohavi, R. (1995). A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection. *Proceedings of IJCAI*, 1137–1143.
9. R Core Team (2026). *R: A Language and Environment for Statistical Computing* (version 4.6.1). R Foundation for Statistical Computing, Vienna, Austria.
10. Ramana, B., & Venkateswarlu, N. (2022). ILPD (Indian Liver Patient Dataset) [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C5D02C>